

UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

Maestría en Analítica de Datos
Machine Learning 2

PRONÓSTICO DE OCUPACIÓN POR BARRIO EN BARCELONA

Informe técnico del proyecto final

Ciudad asignada: Barcelona

Problema 7: Pronóstico de ocupación por barrio

Estudiante:

Atencio Jose 9-710-310
Bosques Karina 8-942-2155
Campos Joshua · 8-890-2410

Profesor: Juan Montenegro, M.Sc.

5 de Octubre de 2026

Índice

Índice de figuras	2
Indice de Tablas	3
1.1. Objetivo general.....	3
1.2. Objetivos específicos.....	4
1.3. Pipeline metodológico del proyecto.....	4
2. Formulación del problema y datos	5
2.1. Fuente, población y construcción del objetivo	5
2.2. Preparación, cobertura y variables.....	5
3. Metodología de validación y baselines	6
3.1. Partición temporal y folds	6
3.2. Baselines y comparación ingenua.....	7
4. Modelos predictivos.....	8
4.1. XGBoost.....	8
4.2. Red neuronal MLP	8
5. Resultados e interpretabilidad.....	9
5.1. Comparación y suficiencia de datos.....	9
5.2. Interpretabilidad, errores y subgrupos.....	10
6. Evaluación final y despliegue	11
6.1. Test final y reproducibilidad	11
6.2. Aplicación web	12
7. Reflexión Crítica final.....	13
7.1. Preparación de los datos	13
7.2. Los dos enfoques.....	14
7.3. La decisión de despliegue	14
7.4. Evaluación de los resultados.....	14
7.5. Reflexión de ética y de uso responsable.....	15
7.6. Limitaciones del Estudio	15
Apendice	16

Índice de figuras

Figura 1: Pipeline metodológico del proyecto	4
Figura 2: Desempeño promedio en validación temporal.	7
Figura 3: Curva de aprendizaje de XGBoost.	10

Figura 4:Comparación de importancia predictiva entre XGBoost y MLP.....	11
Figura 5:Resultado final sobre el conjunto de test.....	12
Figura 6:Flujo técnico del proyecto desde los datos hasta la predicción.	13

Indice de Tablas

Tabla 1: Resumen de los archivos de origen.....	5
Tabla 2: Diccionario de variables utilizadas.....	6
Tabla 3: Esquema de partición temporal.	7
Tabla 4: Resultados promedio en los tres folds de validación temporal.	7
Tabla 5: Configuraciones seleccionadas con el mismo presupuesto de optimización.	9
Tabla 6: MAE por distrito en el Fold 3.....	11
Tabla 7: Resultados finales sobre el conjunto de test.	12
Tabla 8:Comparación técnica de los dos modelos entrenados.....	12

1. Introducción y objetivos

En este proyecto buscamos el pronóstico de ocupación estimada por barrio en Barcelona a partir de datos de Inside Airbnb. El problema fue planteado desde la perspectiva de un inversionista que necesita comparar zonas antes de la compra de una propiedad destinada al alquiler de corta estancia. El trabajo no se enfocó únicamente en obtener una métrica, sino en construir un proceso reproducible, temporalmente correcto y suficientemente explicable para sustentar una decisión.

El desarrollo de este proyecto inicia de la disponibilidad diaria publicada para cada alojamiento. Debido a que el archivo **calendar** no contiene el barrio, fue necesario incorporar información geográfica de **listings** y construir una tasa de ocupación estimada a nivel barrio-fecha. Esta tasa debe interpretarse como una aproximación basada en fechas marcadas como no disponibles; no equivale a reservas confirmadas, ya que un anfitrión también puede bloquear fechas por otras razones.

1.1. Objetivo general

Desarrollar un modelo de Machine Learning que permita pronosticar la tasa de ocupación estimada de alojamientos por barrio en Barcelona, utilizando información de disponibilidad diaria, con el propósito de apoyar la toma de decisiones de un inversionista interesado en identificar zonas con potencial para la adquisición de propiedades destinadas al alquiler de corta estancia.

1.2. Objetivos específicos

- Construir una variable objetivo consistente a partir de la disponibilidad diaria y la ubicación de los alojamientos.
- Aplicar una validación temporal que evite utilizar información futura durante el entrenamiento.
- Comparar un baseline trivial, un baseline estacional, un modelo XGBoost y una red neuronal MLP bajo una métrica común.
- Analizar la suficiencia de los datos, la interpretabilidad, los errores y el desempeño por subgrupos.
- Seleccionar y publicar un modelo en una aplicación web reproducible, acompañando la predicción con una explicación y un rango estimado.

La métrica principal seleccionada fue el **MAE** (Error Medio Absoluto), porque expresa directamente la magnitud promedio del error y evita que errores positivos y negativos se cancelen. Para el interesado del problema resulta más fácil interpretar cuántos puntos de ocupación se desvía, en promedio, el pronóstico.

1.3. Pipeline metodológico del proyecto

La Figura X resume las etapas del proyecto, desde la preparación de los datos hasta la evaluación de los modelos y su implementación en la aplicación Streamlit



Figura 1: Pipeline metodológico del proyecto

2. Formulación del problema y datos

2.1. Fuente, población y construcción del objetivo

Se utilizó la data de Inside Airbnb para Barcelona correspondiente al 24 de junio de 2026. El archivo principal fue **calendar.csv.gz**, complementado con **listings.csv.gz** únicamente para asociar cada **listing_id** con su barrio. Esta incorporación adicional fue necesaria porque calendar contiene la disponibilidad diaria, pero no incluye la variable geográfica requerida para agregar el problema por barrio.

Archivo	Dimensión	Uso principal
calendar.csv.gz	5,623,190 × 5	Disponibilidad diaria, fechas y listing_id.
listings.csv.gz	15,293 × 90	Asignación de neighbourhood_cleansed y distrito.

Tabla 1: Resumen de los archivos de origen.

Calendar contenía 15,406 identificadores de alojamientos, mientras listings contenía 15,293. Se encontraron 15,293 identificadores comunes y 113 presentes únicamente en calendar. Esos 113 representaban 41,245 filas, equivalentes a 0.7335% del calendario. Se decidió excluirlos porque no era posible asignarles un barrio de forma confiable.

La variable neighbourhood no se utilizó porque estaba 100% vacía. En cambio, neighbourhood_cleansed estaba completa y contenía 69 barrios, por lo que fue seleccionada como nivel geográfico. neighbourhood_group_cleansed también se conservó para el análisis posterior por distritos.

La variable objetivo se construyó transformando available = 'f' en unavailable = 1 y available = 't' en unavailable = 0. Luego se calculó el promedio de unavailable por fecha y barrio. De esta forma, occupancy_rate toma valores entre 0 y 1 y representa la proporción de alojamientos marcados como no disponibles en un barrio durante un día.

El contrato del target quedó definido como la **proporción diaria de alojamientos** no disponibles por neighbourhood_cleansed, interpretada como ocupación estimada basada en la disponibilidad publicada. Esta definición permitió mantener el problema medible sin afirmar que cada fecha no disponible corresponde necesariamente a una reserva.

2.2. Preparación, cobertura y variables

Después del cruce se obtuvieron 5,581,945 filas con información geográfica, sin duplicados listing-fecha y sin valores nulos en las variables de ubicación. La primera agregación produjo 25,749 observaciones barrio-fecha, 69 barrios y 374 fechas. Sin embargo, el número de alojamientos por barrio era muy desigual: existían barrios con un solo listing y otros con más de dos mil.

Para reducir la inestabilidad de grupos extremadamente pequeños se adoptó un umbral mínimo de 10 listings. Con este criterio se conservaron 58 barrios, que representaban aproximadamente 99.71% de los alojamientos. También se identificó un período de cobertura completa entre el 3 de julio de 2026 y

el 23 de junio de 2027, con 356 fechas consecutivas. La base final antes de crear rezagos quedó en 20,648 filas, equivalentes a 58 barrios por 356 días.

Variable	Tipo	Significado
occupancy_rate	Objetivo	Proporción estimada de alojamientos no disponibles por barrio y fecha.
lag_7	Numérica	occupancy_rate del mismo barrio exactamente 7 días antes.
lag_14	Numérica	occupancy_rate del mismo barrio exactamente 14 días antes.
lag_28	Numérica	occupancy_rate del mismo barrio exactamente 28 días antes.
day_of_week	Numérica	Día de la semana de la fecha a pronosticar.
month	Numérica	Número de mes de la fecha a pronosticar.
neighbourhood_cleansed	Católica	Barrio de Barcelona.
neighbourhood_group_cleansed	Subgrupo	Distrito utilizado para análisis de desempeño.

Tabla 2: Diccionario de variables utilizadas.

Los lapsos de tiempo se calcularon con shift por barrio, por lo que lag_7 no es un promedio móvil, representa el valor exacto del mismo barrio siete días antes. Al exigir lag_28 se perdieron de forma natural los primeros 28 días de cada barrio. No se imputaron estos valores porque no existía historia suficiente para construirlos. El conjunto de modelado quedó finalmente en 19,024 observaciones, 58 barrios y 328 fechas, desde el 31 de julio de 2026 hasta el 23 de junio de 2027.

El horizonte de trabajo se fijó en 28 días y se adoptó una evaluación rolling, para predecir una fecha solo se utilizan valores observados en fechas anteriores. Los lapsos de tiempo fueron la principal forma de representar la estacionalidad semanal sin introducir información futura.

3. Metodología de validación y baselines

3.1. Partición temporal y folds

La separación de los datos se diseñó antes de evaluar los modelos finales. **Se reservaron los últimos 28 días como test**, desde el 27 de mayo hasta el 23 de junio de 2027. Este conjunto contiene 1,624 observaciones y no participó en la selección de hiperparámetros, la comparación de configuraciones ni el Early Stopping.

Los datos para desarrollo quedaron formados por 17,400 observaciones y 300 fechas, desde el 31 de julio de 2026 hasta el 26 de mayo de 2027. Sobre este período se construyeron tres folds temporales expansivos. Cada fold utiliza únicamente fechas anteriores para entrenamiento y las siguientes 28 fechas para validación.

Fold	Entrenamiento	Filas train	Validación	Filas val
1	31-jul-2026 a 03-mar-2027	12,528	04-mar a 31-mar-2027	1,624
2	31-jul-2026 a 31-mar-2027	14,152	01-abr a 28-abr-2027	1,624
3	31-jul-2026 a 28-abr-2027	15,776	29-abr a 26-may-2027	1,624
Test final	31-jul-2026 a 26-may-2027	17,400	27-may a 23-jun-2027	1,624

Tabla 3: Esquema de partición temporal.

Esta estructura fue importante porque una partición aleatoria permitiría que fechas futuras formen parte del entrenamiento utilizado para evaluar fechas anteriores. Aunque el modelo no vea explícitamente el target de la fila de validación, el orden temporal del problema quedaría roto. Por esa razón la validación temporal se mantuvo como criterio metodológico incluso cuando la partición aleatoria no produjo una métrica artificialmente mejor.

Todo el preprocesamiento específico de cada modelo se mantuvo dentro de su flujo de entrenamiento. XGBoost utilizó passthrough para las variables numéricas y OneHotEncoder para barrio. El MLP aplicó StandardScaler a las variables numéricas y OneHotEncoder a la variable categórica, ajustando ambos transformadores únicamente con la porción de entrenamiento de cada fold.

3.2. Baselines y comparación ingenua

Se establecieron dos referencias antes de interpretar los modelos complejos. El **baseline trivial** utilizó la media del target de entrenamiento y obtuvo un MAE promedio de 0.08955. El **baseline estacional** utilizó directamente lag_7 como predicción y alcanzó 0.01735. Este segundo resultado fue especialmente exigente ya que confirmó que la semanal era muy fuerte y estableció un piso difícil de superar.

Enfoque	MAE validación
Baseline media	0.08955
Baseline estacional lag_7	0.01735
XGBoost base	0.02442
XGBoost optimizado	0.02169
MLP base	0.02586
MLP optimizado	0.02448

Tabla 4: Resultados promedio en los tres folds de validación temporal.

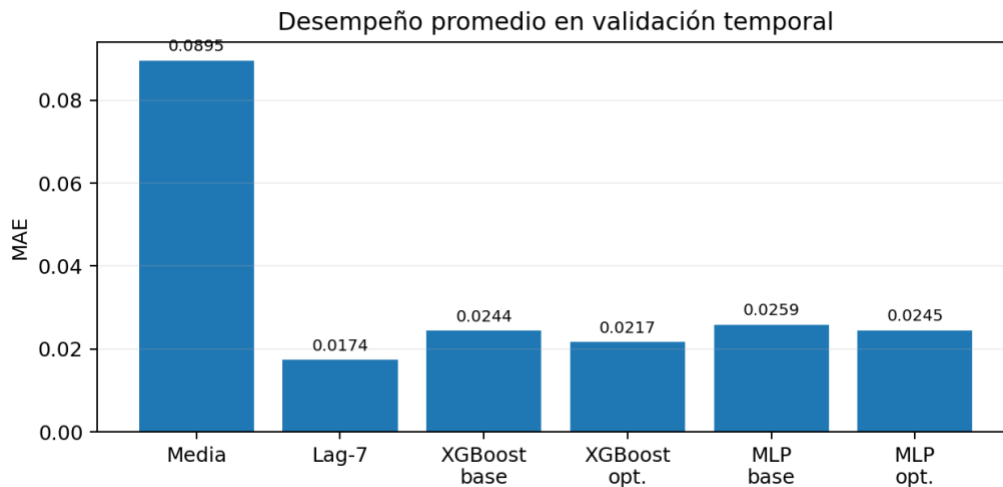


Figura 2: Desempeño promedio en validación temporal.

El baseline lag_7 mejoró aproximadamente 80.6% frente al baseline de la media. Ninguno de los dos modelos entrenados logró superarlo durante validación. Este resultado no se consideró un fracaso del proyecto; al contrario, mostró que una regla estacional sencilla puede ser muy competitiva cuando existe una periodicidad semanal marcada.

También se comparó XGBoost optimizado bajo una partición aleatoria equivalente a 28/300 del Development. El MAE aleatorio fue 0.022998 frente a 0.021687 de la validación temporal. En este caso, la partición ingenua no produjo una métrica más optimista. Aun así, se descartó porque no representa cómo se utilizaría el modelo para pronosticar períodos futuros y permite mezclar cronológicamente entrenamiento y validación.

4. Modelos predictivos

4.1. XGBoost

El modelo clásico seleccionado fue XGBoost mediante XGBRegressor. Se eligió boosting porque el problema es tabular y contiene relaciones no lineales entre rezagos, calendario y barrio. Las variables numéricas se pasaron sin escalamiento, ya que los árboles no requieren normalización para construir sus divisiones, mientras que el barrio se transformó mediante OneHotEncoder con manejo de categorías desconocidas.

La configuración base obtuvo un MAE promedio de 0.024419. Para la optimización se fijó previamente un presupuesto de 10 configuraciones evaluadas sobre los mismos tres folds temporales, equivalente a 30 entrenamientos. La configuración ganadora utilizó 300 árboles, profundidad máxima 3, learning_rate 0.05 y subsample 0.80.

Con estos hiperparámetros, los MAE por fold fueron 0.030756, 0.016414 y 0.017890, con un promedio de 0.021687. La mejora respecto al XGBoost base fue cercana a 11.2%. El resultado seguía por encima del baseline lag_7, pero representó el mejor desempeño entre los dos modelos aprendidos.

La optimización no se amplió después de observar los resultados. Mantener el presupuesto fijado fue importante para que la comparación con el MLP no se convirtiera en una búsqueda indefinida a favor de uno de los modelos.

Desde el punto de vista operativo, XGBoost también ofreció una ventaja importante: se pudo guardar el Pipeline completo, incluyendo el preprocesamiento categórico. Esto simplificó la aplicación web, porque la entrada del usuario puede enviarse directamente al Pipeline sin reproducir manualmente la codificación del barrio.

4.2. Red neuronal MLP

El segundo enfoque fue un perceptrón multicapa implementado en Keras. A diferencia de XGBoost, el MLP requirió escalar las variables numéricas. El preprocesamiento produjo aproximadamente 63 características: cinco numéricas y las columnas one-hot correspondientes a los 58 barrios.

El modelo base utilizó dos capas ocultas de 64 y 32 neuronas con activación ReLU, dropout de 0.20 y una salida lineal. Se entrenó con Adam, función de pérdida MAE y Early Stopping con paciencia de 15 épocas y restauración de los mejores pesos. Su MAE promedio fue 0.025861.

La optimización utilizó el mismo presupuesto que XGBoost: 10 configuraciones por tres folds. Se variaron arquitectura, dropout, learning rate y batch size. La mejor configuración fue más simple que la arquitectura base: una sola capa oculta de 64 neuronas, dropout 0.10, learning rate 0.001 y batch size 64.

Modelo	Configuración seleccionada	MAE validación
XGBoost	300 árboles; depth=3; lr=0.05; subsample=0.80	0.021687
MLP	(64); dropout=0.10; lr=0.001; batch=64	0.024476

Tabla 5: Configuraciones seleccionadas con el mismo presupuesto de optimización.

El MLP optimizado obtuvo MAE por fold de 0.030066, 0.025449 y 0.017913, para un promedio de 0.024476. La mejora respecto al MLP base fue cercana a 5.4%, pero no alcanzó el desempeño de XGBoost.

Para estudiar el sobreajuste se volvieron a ejecutar los tres folds únicamente para capturar las curvas de pérdida, sin utilizar esos resultados para cambiar la configuración seleccionada. Las mejores épocas observadas fueron 11, 7 y 36. En las curvas, la pérdida de entrenamiento continuaba disminuyendo mientras la validación se estabilizaba o fluctuaba, señal del inicio de sobreajuste. Early Stopping permitió detener el entrenamiento y restaurar los pesos asociados al mejor punto de validación.

5. Resultados e interpretabilidad

5.1. Comparación y suficiencia de datos

La comparación de validación mostró tres conclusiones claras. Primero, el baseline estacional lag_7 fue el enfoque con menor MAE. Segundo, XGBoost fue superior al MLP entre los modelos entrenados. Tercero, la red neuronal no justificó una mejora en exactitud frente al boosting, por lo que su mayor complejidad no se tradujo en una ventaja predictiva.

La suficiencia de datos se evaluó con una curva de aprendizaje de XGBoost usando el Fold 3 como período fijo de validación. Se entrenó con 10%, 25%, 50%, 75% y 100% de las fechas de entrenamiento, tomando en cada caso las fechas más recientes disponibles para preservar el orden temporal.

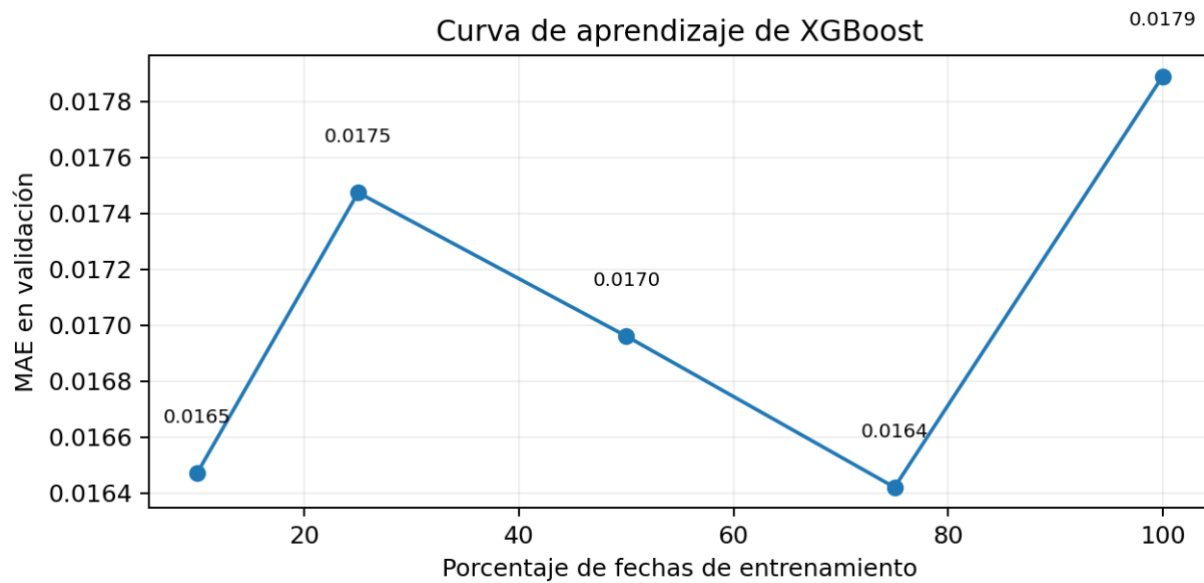


Figura 3: Curva de aprendizaje de XGBoost.

Los MAE fueron 0.016472, 0.017474, 0.016961, 0.016421 y 0.017890, respectivamente. La curva no mejoró de manera monotonica y el 100% no fue el mejor punto. Por tanto, no se encontró evidencia de que simplemente agregar más historia de la misma forma garantizara una mejora. Tampoco se utilizó el 75% como una nueva decisión de tuning, porque hacerlo después de observar la curva habría convertido el análisis de suficiencia en una optimización adicional no planificada.

El resultado sugiere que la relevancia temporal de la historia puede ser tan importante como el volumen. Sin embargo, no se atribuyó una causa definitiva a esta variación porque el experimento no fue diseñado para aislar cambios de régimen o estacionalidad.

5.2. Interpretabilidad, errores y subgrupos

La interpretabilidad global se evaluó mediante permutation importance sobre las seis variables originales. En ambos modelos, lag_7 fue claramente la señal dominante. XGBoost mostró una dependencia secundaria mayor de lag_14, mientras el MLP mostró mayor sensibilidad a la variable de barrio. lag_28 tuvo una contribución menor y month y day_of_week resultaron prácticamente nulos en esta evaluación global.

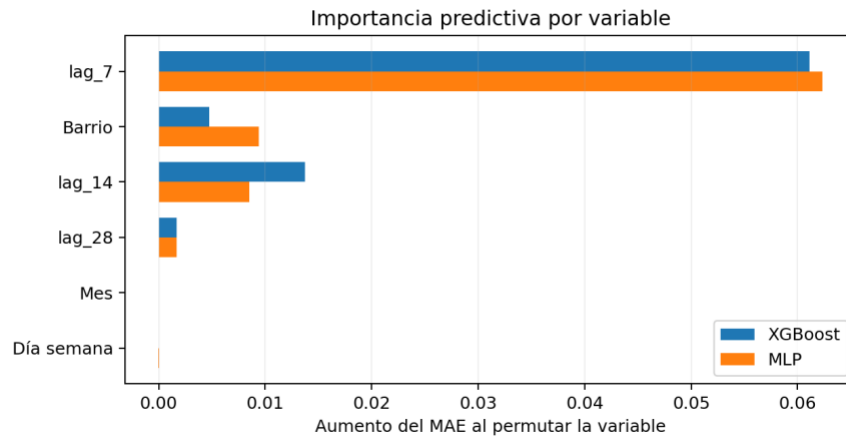


Figura 4: Comparación de importancia predictiva entre XGBoost y MLP.

Estas importancias se interpretan como dependencia predictiva, no como causalidad. Una variable importante ayuda al modelo a predecir; no demuestra que modificar esa variable provoque un cambio real en la ocupación.

Los cinco mayores errores de XGBoost y MLP se concentraron en los Roquetes entre el 3 y el 7 de mayo de 2027. En ese período la ocupación estimada cayó de forma abrupta desde valores cercanos a 46.15% hasta 15.38%, mientras los rezagos todavía reflejaban valores anteriores altos. El barrio tenía 13 listings, por lo que cada alojamiento representaba aproximadamente 7.69 puntos porcentuales de la tasa agregada. El patrón mostró una limitación común: los rezagos reaccionan con retraso ante cambios bruscos.

Distrito	XGBoost	MLP
Sarrià-Sant Gervasi	0.030644	0.028824
Nou Barris	0.030085	0.030793
Les Corts	0.028346	0.036144
Gràcia	0.014964	0.015796
Sant Martí	0.014623	0.012009
Eixample	0.014417	0.017526
Horta-Guinardó	0.014336	0.014072
Sant Andreu	0.013611	0.014560
Sants-Montjuïc	0.013607	0.010844
Ciutat Vella	0.007651	0.012234

Tabla 6: MAE por distrito en el Fold 3.

6. Evaluación final y despliegue

6.1. Test final y reproducibilidad

Una vez cerradas las 11 decisiones de modelado se entrenaron los modelos finales con las 17,400 observaciones de Development. Para el MLP se fijaron 13 épocas antes de abrir el test,

correspondientes a la mediana de las mejores épocas de la optimización original (16, 6 y 13). De esta forma, el test no se ecisio para Early Stopping ni para ajustar ecisions.1212

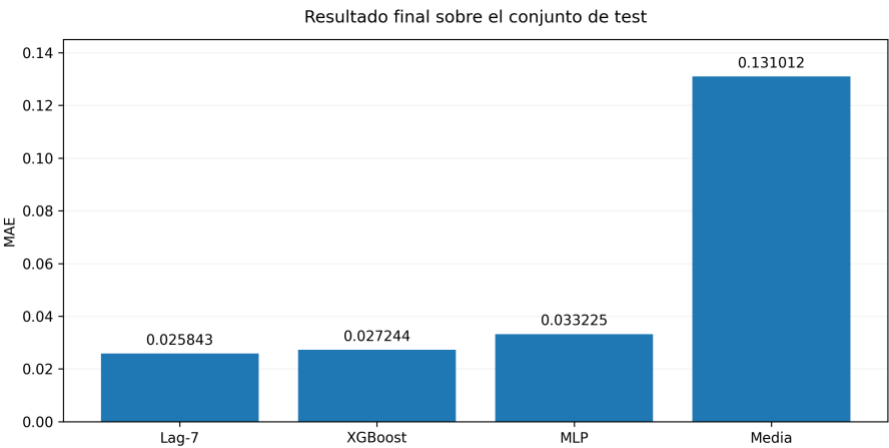


Figura 5:Resultado final sobre el conjunto de test.

El orden observado en validación se mantuvo: lag_7 obtuvo el menor MAE, seguido de XGBoost, MLP y la media. Los errores aumentaron en el período final para todos los enfoques, pero no se modificó ningún hiperparámetro después de observar el test. A partir de este punto el test se consideró cerrado.

Modelo	MAE	Train (s)	Inferencia (s)	Disco (KB)
XGBoost	0.027244	0.2333	0.010497	346.12
MLP	0.033225	1.4542	0.074043	75.11

Tabla 7: Comparación técnica de los dos modelos entrenados.

XGBoost fue más preciso y más rápido que el MLP en entrenamiento e inferencia, aunque ocupó más espacio en disco. El MLP fue aproximadamente 6.2 veces más lento al entrenar y 7.1 veces más lento al inferir en este experimento. Estas cifras describen el equipo y la implementación utilizados; no se presentan como propiedades universales de los algoritmos.

Los artefactos guardados fueron cargados nuevamente y reprodujeron exactamente las predicciones originales. XGBoost y MLP mostraron diferencia máxima 0.0 frente a sus predicciones previas. También se reprodujeron los baselines lag_7 y media, fortaleciendo la trazabilidad del proyecto.

6.2. Aplicación web

Para el despliegue se seleccionó XGBoost como modelo aprendido de producción. Aunque lag_7 obtuvo el mejor MAE global, XGBoost fue el mejor de los dos modelos entrenados exigidos por el proyecto y permite integrar simultáneamente rezagos, calendario y barrio dentro de un Pipeline reproducible. Esta decisión se documenta reconociendo de forma explícita que el baseline estacional fue superior en exactitud; no se ocultó esa comparación.

La aplicación fue desarrollada con Streamlit y publicada mediante Streamlit Community Cloud. El usuario selecciona el barrio, la fecha a pronosticar y los porcentajes de ocupación estimada observados 7, 14 y 28 días antes. La aplicación deriva month y day_of_week, construye las seis variables requeridas y utiliza el Pipeline XGBoost guardado para generar la predicción.

Para evitar presentar el pronóstico como una cifra exacta, se añadió un rango estimado a partir de 4,872 errores fuera de muestra obtenidos en los tres folds de validación temporal. El percentil 90 del error absoluto fue 0.050693, equivalente a aproximadamente 5.07 puntos porcentuales. La aplicación muestra la predicción ± 5.07 puntos, limitada al rango natural de 0% a 100%. El test final no se utilizó para calibrar este rango.

La explicación local se implementó con SHAP TreeExplainer. Las contribuciones de las columnas one-hot de barrio se agrupan nuevamente como un único factor para que la visualización sea comprensible. Un valor SHAP positivo indica que la variable empuja esa predicción hacia arriba respecto al valor base del explainer; uno negativo la reduce. Estas contribuciones explican el comportamiento del modelo y no deben interpretarse como causalidad.

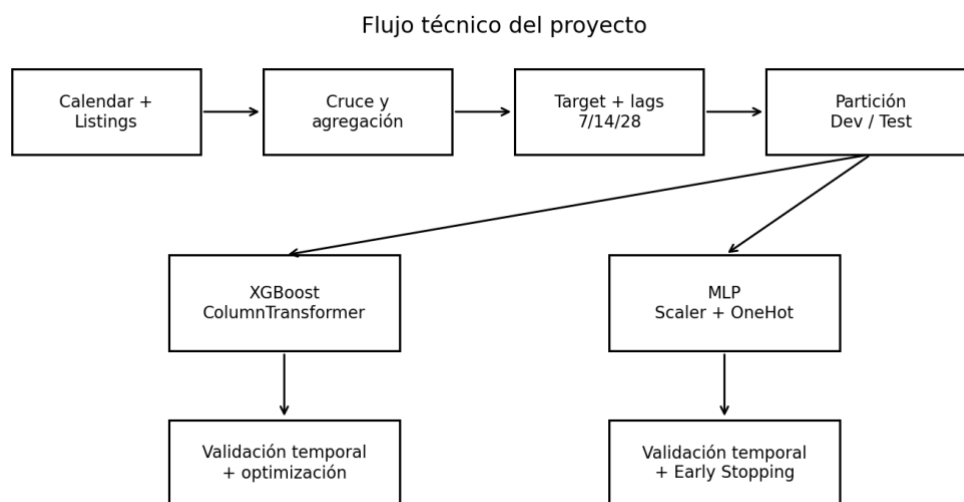


Figura 6:Flujo técnico del proyecto desde los datos hasta la predicción.

La URL pública de la aplicación es: <https://barcelona-occupancy-ml2.streamlit.app>. El repositorio incluye app/app.py, el modelo XGBoost, los artefactos de los demás enfoques, requirements.txt con versiones fijadas y README con instrucciones de ejecución. La aplicación fue probada tanto localmente como desde la URL pública.

7. Reflexión Crítica final

7.1. Preparación de los datos

¿Qué transformaciones exigió el problema asignado antes de poder modelar? ¿Qué parte costó más trabajo?

Antes de modelar fue necesario construir la unidad de análisis. El archivo calendar contenía solo la disponibilidad con el código de alojamiento, por lo que se necesitaba obtener el barrio del archivo de listings. De igual forma, se tuvo que transformar la variable available en una medida binaria. Y para

obtener la tasa de ocupación, se agruparon estos datos a nivel de barrio – fecha. La parte más importante fue asegurar que la variable objetivo realmente representara la ocupación por barrio y mantener el sentido con el enunciado del problema.

¿Qué encontraron sucio, incompleto o inconsistente, y qué decidieron hacer al respecto?

La variable de neighbourhood estaba contenía solo nulos y no podía utilizarse, utilizamos un campo neighbourhood_cleansed que contenía los 69 barrios. Existían 113 alojamientos en calendar que no estaban registrados en listings, excluimos estos alojamientos ya que no podríamos asignarlos a un barrio. Había barrios con muy pocos alojamientos, por lo que mantuvimos aquellos con más de 10 para mejorar la estabilidad del modelo.

¿Por qué ese target y esa métrica, dado el interesado que plantea el enunciado?

Utilizamos como la ocupación por barrios, que fue calculada utilizando la variable available transformada. Esta variable responde directamente la pregunta del inversionista. La métrica que utilizamos fue MAE (error absoluto medio) porque se expresa en las mismas unidades que la ocupación y no permite que los errores positivos y negativos se cancelen, por lo que es fácil de interpretar.

7.2. Los dos enfoques

¿Cuánto mejoró la red neuronal respecto al modelo clásico? ¿La mejora justificó el costo adicional?

La red neuronal no mejoró con respecto al modelo clásico que utilizamos. El modelo XGBoost tuvo 0.002789 unidades menos que la red neuronal y el modelo clásico fue más rápido al entrenar e inferir. Por lo que la mejora no fue justificada porque obtuvo menor exactitud y mayores costos computacionales.

¿En qué se apoya cada modelo para decidir? Relacionar con la idea de ingeniería de características frente a aprendizaje de representaciones.

Ambos modelos se apoyaron con las características de rezagos temporales, siendo la más importante lag_7. XGboost depende de estas variables para poder predecir, lo cual confirmamos con los valores SHAP. Mientras que MLP aprendió los patrones con estas variables creadas.

7.3. La decisión de despliegue

¿Cuál de los tres pondrías en producción y por qué? Considerar exactitud, costo de inferencia, mantenimiento e interpretabilidad.

Si consideráramos solo la precisión, el modelo de lag_7 sería el mejor. Sin embargo, al considerar todos los demás puntos y tomando en cuenta el objetivo del problema, el óptimo fue el XGBoost. Este fue el mejor de los modelos aprendidos, tiene buena interpretabilidad, es rápido en entrenamiento e inferencia y permite incorporar nuevas variables sin modificar la lógica.

7.4. Evaluación de los resultados

¿Qué errores comete el modelo elegido? ¿Hay algún patrón en ellos?

El modelo XGBoost, presentó mayores errores frente a cambios bruscos en la serie temporal. Además, en barrios pequeños, el porcentaje de ocupación puede verse elevado por lo que pequeños cambios pueden verse más grande de forma porcentual.

¿Qué diferencia hubo entre la partición ingenua y la correcta? ¿Qué enseña esa diferencia?

La diferencia entre ambos fue pequeña, y la partición ingenua no supero a la correcta al compararla con la métrica. Nos enseña, que la decisión debe basarse en la naturaleza del problema y no en cuál nos da el mejor resultado.

7.5. Reflexión de ética y de uso responsable

¿Qué consecuencias tendría este modelo mal usado por el interesado que plantea el enunciado?

Si el inversionista interpreta las predicciones con certeza, podría realizar compras en zonas con menor potencial real o asumir riesgos financieros incorrectos. Además, la variable target representa una ocupación estimada en disponibilidad publicada y no reservas reales. Con respecto a la privacidad, el modelo solo solicita variables relacionadas con barrio, fecha y ocupación histórica. No solicita datos personales de personas. De igual forma, los datos que se guardan con joblib contienen solo los parámetros del modelo, no almacenan información personal.

¿Qué encontraron al comparar la métrica entre subgrupos? ¿A quién perjudicaría si se desplegara así?

Observamos algunas diferencias importantes al comparar la métrica entre grupos. El modelo no tiene la misma precisión para todos los grupos, en ciertos grupos el error es mayor. Evaluar únicamente el error global oculta este tipo de diferencias importantes, sesgando diferentes zonas. Al final, la responsabilidad de las decisiones tomadas recae en el interesado que utiliza esta información. Recordando que el modelo debe tomarse como una herramienta de apoyo y no sustituto del juicio humano.

7.6. Limitaciones del Estudio

¿Qué limitaciones tuvieron los datos, el tiempo de cómputo disponible o el diseño de experimento?

Los datos no contienen reservas confirmadas y es una foto de Inside Airbnb, además que algunos barrios poseen pocos alojamientos y algunos alojamientos no cuentan con data geográfica. Por otro lado, el tiempo de cómputo utiliza el mismo costo de configuración (fue limitado a 10 por modelo) y el diseño experimental no utiliza factores externos como clima, precios, eventos. La evaluación se realizó con una solo foto temporal del mercado.

¿Qué harían distinto con tres meses más y una GPU dedicada?

Con tres meses adicionales y mayor GPU, podríamos realizar varias mejoras. Se podría recopilar varias fotos de datos y construir una serie temporal más realista, también incorporar variables externas importantes para la interpretación de cambios bruscos en la serie temporal. De igual manera, se podría evaluar modelos más complejos y aumentar el costo de configuraciones.

Apendice

Codigo Fuente del Proyecto

=====

BLOQUE DE CÓDIGO 1

=====

LIBRERÍAS Y CONFIGURACIÓN GENERAL

Utilidades

```
from pathlib import Path import os import time import joblib
```

Datos y gráficos

```
import numpy as np import pandas as pd import matplotlib.pyplot as plt
```

Scikit-learn

```
from sklearn.base import clone from sklearn.compose import ColumnTransformer from
sklearn.inspection import permutation_importance from sklearn.metrics import
mean_absolute_error from sklearn.model_selection import train_test_split from
sklearn.pipeline import Pipeline from sklearn.preprocessing import OneHotEncoder,
StandardScaler
```

XGBoost

```
import xgboost as xgb from xgboost import XGBRegressor
```

Redes neuronales

```
import tensorflow as tf from keras.models import Sequential from keras.layers import Dense,
Dropout, Input from keras.optimizers import Adam from keras.callbacks import EarlyStopping
from keras.utils import set_random_seed
```

Semilla para reproducibilidad

```
SEED = 42
```

```
np.random.seed(SEED) tf.random.set_seed(SEED) set_random_seed(SEED)
```

```
tf.config.experimental.enable_op_determinism()
```

=====

BLOQUE DE CÓDIGO 2

=====

Rutas relativas del proyecto

```
PROJECT_PATH = Path(".") RAW_PATH = PROJECT_PATH / "data" / "raw" PROCESSED_PATH =
PROJECT_PATH / "data" / "processed" MODELS_PATH = PROJECT_PATH / "models" FIGURES_PATH =
PROJECT_PATH / "figures"
```

```
#print("RAW:", RAW_PATH.resolve())
```

Cargamos nuestros Datos desde los archivos fuentes.abs

```

calendar = pd.read_csv(RAW_PATH / "calendar.csv.gz") listings = pd.read_csv(RAW_PATH /
"listings.csv.gz")

print("Archivos cargados correctamente.")

=====

BLOQUE DE CÓDIGO 3

=====

Verificamos cantidad de Registros y columnas de cada archivo.

print("CALENDAR") print("Filas:", calendar.shape[0]) print("Columnas:", calendar.shape[1])

print("\nLISTINGS") print("Filas:", listings.shape[0]) print("Columnas:",
listings.shape[1])

=====

BLOQUE DE CÓDIGO 4

=====

Visualizamos las columnas en cada archivo

print("***** COLUMNAS DE CALENDAR:***** ") print(calendar.columns.tolist())

print("\n***** COLUMNAS DE LISTINGS:***** ") print(listings.columns.tolist())

=====

BLOQUE DE CÓDIGO 5

=====

print("\nTipos de datos:") print(calendar.dtypes)

print("\nValores nulos:") print(calendar.isnull().sum())

print("\nValores únicos de 'available':")
print(calendar["available"].value_counts(dropna=False))

print("\nNúmero de alojamientos únicos:") print(calendar["listing_id"].nunique())

print("\nRango de fechas:") print("Desde:", calendar["date"].min()) print("Hasta:",
calendar["date"].max())

print("\nFilas duplicadas:") print(calendar.duplicated().sum())

print("\nDuplicados listing_id + date:") print(calendar.duplicated(subset=["listing_id",
"date"]).sum())

=====

BLOQUE DE CÓDIGO 6

=====

print("\nFilas:", len(listings)) print("IDs únicos:", listings["id"].nunique()) print("IDs
duplicados:", listings["id"].duplicated().sum()) print("IDs nulos:",
listings["id"].isnull().sum())

=====

```

BLOQUE DE CÓDIGO 7

```
=====

calendar_ids = set(calendar["listing_id"].unique()) listings_ids =
set(listings["id"].unique())

solo_calendar = calendar_ids - listings_ids solo_listings = listings_ids - calendar_ids
comunes = calendar_ids & listings_ids

print("\nIDs en Calendar:", len(calendar_ids)) print("IDs en Listings:", len(listings_ids))
print("IDs comunes:", len(comunes)) print("Solo en Calendar:", len(solo_calendar))
print("Solo en Listings:", len(solo_listings))

=====
```

BLOQUE DE CÓDIGO 8

```
=====

filas_sin_listing = calendar[ calendar["listing_id"].isin(solo_calendar) ]
print("\nAlojamientos afectados:", filas_sin_listing["listing_id"].nunique())

print("Filas afectadas:", len(filas_sin_listing))

print("Porcentaje de filas afectadas:", round(len(filas_sin_listing) / len(calendar) * 100,
4), "%")

=====
```

BLOQUE DE CÓDIGO 9

```
=====

columnas_geo = [ "id", "neighbourhood", "neighbourhood_cleansed",
"neighbourhood_group_cleansed" ]

for col in columnas_geo: print(f"\n{col}") print("Nulos:", listings[col].isnull().sum())
print("Valores únicos:", listings[col].nunique())

=====
```

BLOQUE DE CÓDIGO 10

```
=====

print ("\n") barrios = ( listings["neighbourhood_cleansed"] .value_counts()
.sort_values(ascending=False) )

print(barrios)

=====
```

BLOQUE DE CÓDIGO 11

```
=====

print ("\n")

distritos = ( listings["neighbourhood_group_cleansed"] .value_counts()
.sort_values(ascending=False) )

print(distritos)
```

```
=====
BLOQUE DE CÓDIGO 12
=====
```

```
print("\n=== TOP 20: neighbourhood_cleansed ===")
print(listings["neighbourhood_cleansed"].value_counts().head(20))

print("\n=== neighbourhood_group_cleansed ===")
print(listings["neighbourhood_group_cleansed"].value_counts())

=====
```

```
BLOQUE DE CÓDIGO 13
=====
```

```
#Validar el tamaño de disponibilidad total de los barrios para identificar si hay barrios
con muy poca ocupacion #que nos pueda sesgar los resultados conteo_barrios = (
listings["neighbourhood_cleansed"] .value_counts() .sort_values() )

print("=== DISTRIBUCIÓN DE LISTINGS POR BARRIO ===")

print("Número de barrios:", len(conteo_barrios)) print("Mínimo de listings:",
conteo_barrios.min()) print("Máximo de listings:", conteo_barrios.max()) print("Mediana:",
conteo_barrios.median())

print("\n=== 15 BARRIOS CON MENOS LISTINGS ===") print(conteo_barrios.head(15))

=====
```

```
BLOQUE DE CÓDIGO 14
=====
```

```
umbrales = [1, 5, 10, 20, 30, 50, 100]

resultados_umbral = []

for minimo in umbrales:

    barrios_validos = conteo_barrios[
        conteo_barrios >= minimo
    ]

    listings_incluidos = barrios_validos.sum()

    resultados_umbral.append({
        "minimo_listings": minimo,
        "barrios conservados": len(barrios_validos),
        "cobertura": listings_incluidos / len(listings) * 100
    })

df_umbrales = pd.DataFrame(resultados_umbral)

df_umbrales

=====
```

```
BLOQUE DE CÓDIGO 15
=====
```

```

total_barrios = 69

df_grafico = df_umbrales.sort_values( "minimo_listings", ascending=False ).copy()

Calcular barrios eliminados

df_grafico["barrios_eliminados"] = ( total_barrios - df_grafico["barrios conservados"] )

Crear etiqueta del eje Y

df_grafico["etiqueta"] = ( "≥ " + df_grafico["minimo_listings"].astype(str) + " listings ("
+ df_grafico["barrios_eliminados"].astype(str) + " barrios eliminados)" )

Crear gráfico

plt.figure(figsize=(12, 6))

bars = plt.barh( df_grafico["etiqueta"], df_grafico["barrios conservados"] )

Mostrar porcentaje de listings conservados

for bar, cobertura in zip(bars, df_grafico["cobertura"]): plt.text( bar.get_width() + 0.5,
bar.get_y() + bar.get_height() / 2, f"{cobertura:.2f}% listings", va="center", fontsize=10
)

plt.title("Impacto del mínimo de listings por barrio", fontsize=14) plt.xlabel("Número de
barrios conservados") plt.ylabel("Umbral mínimo")

plt.xlim(0, 90) plt.grid(axis="x", alpha=0.3)

plt.tight_layout() plt.show()

```

=====

BLOQUE DE CÓDIGO 16

=====

```

Crear copia de trabajo para no modificar el dataset original

calendar_work = calendar.copy()

Convertir fecha a formato datetime

calendar_work["date"] = pd.to_datetime( calendar_work["date"] )

Transformar disponibilidad:

t = disponible -> 0

f = no disponible -> 1

calendar_work["unavailable"] = ( calendar_work["available"] .map({"t": 0, "f": 1}) )

calendar_work[ ["listing_id", "date", "available", "unavailable"] ].head(10)

```

=====

BLOQUE DE CÓDIGO 17

=====

```

#Validacion de como quedo la dat

```

```

print("\nValores originales de available:")
print(calendar_work["available"].value_counts(dropna=False))

print("\nValores generados de unavailable:")
print(calendar_work["unavailable"].value_counts(dropna=False))

print("\nNulos generados:") print(calendar_work["unavailable"].isnull().sum())

print("\nTipo de date:") print(calendar_work["date"].dtype)

print("\nRango temporal:") print( calendar_work["date"].min(), "→",
calendar_work["date"].max() )

```

=====

BLOQUE DE CÓDIGO 18

```

=====

listings_geo = listings[ [ "id", "neighbourhood_cleaned", "neighbourhood_group_cleaned" ]
].copy()

listings_geo.head()

```

=====

BLOQUE DE CÓDIGO 19

```

=====

validacion_geo = ( listings_geo.groupby("neighbourhood_cleaned")
["neighbourhood_group_cleaned"] .nunique() )

print("Máximo número de grupos asociados a un barrio:") print(validacion_geo.max())

print("\nBarrios asociados a más de un grupo:") print(validacion_geo[validacion_geo > 1])

```

=====

BLOQUE DE CÓDIGO 20

```

=====

Hacemos la integracion.

calendar_geo = calendar_work.merge(listings_geo, left_on="listing_id", right_on="id",
how="inner", validate="many_to_one" )

print("\nFilas calendar original:", len(calendar_work)) print("Filas después del merge:",
len(calendar_geo)) print("Filas eliminadas:", len(calendar_work) - len(calendar_geo))

print("\nListings únicos:") print(calendar_geo["listing_id"].nunique())

print("\nNulos en barrio:") print(calendar_geo["neighbourhood_cleaned"].isnull().sum())

print("\nNulos en grupo:")
print(calendar_geo["neighbourhood_group_cleaned"].isnull().sum())

print("\nDuplicados listing-fecha:") print( calendar_geo.duplicated( subset=["listing_id",
"date"] ).sum() )

```

=====

BLOQUE DE CÓDIGO 21

```
=====

daily_neighbourhood = ( calendar_geo.groupby( [ "date", "neighbourhood_group_cleansed",
"neighbourhood_cleansed" ], as_index=False ).agg( occupancy_rate=("unavailable", "mean"),
n_listings=("listing_id", "nunique") ) )

daily_neighbourhood.head(10)

=====
```

BLOQUE DE CÓDIGO 22

```
=====

print("=== VALIDACIÓN DE OCCUPANCY_RATE ===")

print("\nDimensiones:") print(daily_neighbourhood.shape)

print("\nNúmero de barrios:") print(
daily_neighbourhood["neighbourhood_cleansed"].nunique() )

print("\nNúmero de fechas:") print( daily_neighbourhood["date"].nunique() )

print("\nRango temporal:") print( daily_neighbourhood["date"].min(), "→",
daily_neighbourhood["date"].max() )

print("\nRango de occupancy_rate:") print( daily_neighbourhood["occupancy_rate"].min(),
"→", daily_neighbourhood["occupancy_rate"].max() )

print("\nNulos en occupancy_rate:") print(
daily_neighbourhood["occupancy_rate"].isnull().sum() )

print("\nListings por observación barrio-fecha:") print(
daily_neighbourhood["n_listings"].describe() )

print("\nDuplicados fecha-barrio:") print( daily_neighbourhood.duplicated( subset=["date",
"neighbourhood_cleansed"] ).sum() )

=====
```

BLOQUE DE CÓDIGO 23

```
=====

cobertura_barrios = ( daily_neighbourhood.groupby("neighbourhood_cleansed") .agg(
n_fechas=("date", "nunique"), fecha_inicio=("date", "min"), fecha_fin=("date", "max"),
n_listings_min=("n_listings", "min"), n_listings_max=("n_listings", "max") )
.sort_values("n_fechas") )

print("=== COBERTURA TEMPORAL POR BARRIO ===")

print(cobertura_barrios.head(15))

print("\nDistribución del número de fechas:") print( cobertura_barrios["n_fechas"]
.value_counts() .sort_index() )

=====
```

BLOQUE DE CÓDIGO 24

```
=====

barrios_incompletos = cobertura_barrios[ cobertura_barrios["n_fechas"] < 374 ]
```

```

print("=== BARRIOS CON COBERTURA INCOMPLETA ===") print(barrios_incompletos)

print( "\nNúmero de barrios con menos de 374 fechas:", len(barrios_incompletos) )

=====

BLOQUE DE CÓDIGO 25

=====

VALIDACIÓN DE COBERTURA DE LISTINGS POR FECHA

coverage_date = ( calendar_geo.groupby("date")["listing_id"] .nunique()
.rename("n_listings_total") .reset_index() )

max_listings = coverage_date["n_listings_total"].max()

coverage_date["coverage_pct"] = ( coverage_date["n_listings_total"] / max_listings * 100 )

print("=== RESUMEN DE LISTINGS POR FECHA ===")
print(coverage_date["n_listings_total"].describe())

print("\n=== PRIMERAS 10 FECHAS ===") print(coverage_date.head(10))

print("\n=== ÚLTIMAS 20 FECHAS ===") print(coverage_date.tail(20))

=====

BLOQUE DE CÓDIGO 26

=====

=====

CONSTRUCCIÓN DEL DATASET CON POBLACIÓN DEFINIDA

=====

FECHA_INICIO = pd.Timestamp("2026-07-03") FECHA_FIN = pd.Timestamp("2027-06-23")
MIN_LISTINGS = 10

Barrios con al menos 10 listings

barrios_seleccionados = conteo_barrios[ conteo_barrios >= MIN_LISTINGS ].index

Aplicar ventana temporal + población

df_model = daily_neighbourhood[ (daily_neighbourhood["date"] >= FECHA_INICIO) &
(daily_neighbourhood["date"] <= FECHA_FIN) &
(daily_neighbourhood["neighbourhood_cleansed"].isin( barrios_seleccionados )) ].copy()

print("=== DATASET PARA MODELADO ===") print("Dimensiones:", df_model.shape) print(
"Barrios:", df_model["neighbourhood_cleansed"].nunique() ) print( "Fechas:",
df_model["date"].nunique() ) print( "Rango temporal:", df_model["date"].min(), "-",
df_model["date"].max() ) print( "Nulos occupancy_rate:",
df_model["occupancy_rate"].isna().sum() )

print( "Duplicados fecha-barrio:", df_model.duplicated( subset=["date",
"neighbourhood_cleansed"] ).sum() ) print("\n=== LISTINGS POR OBSERVACIÓN ===") print(
df_model["n_listings"].describe() ) print("\n=== RANGO DEL TARGET ===") print(
df_model["occupancy_rate"].describe() )

=====

```

BLOQUE DE CÓDIGO 27

```
=====
```

PARTICIÓN TEMPORAL: DESARROLLO Y TEST FINAL

HORIZONTE = 28

Fechas únicas ordenadas

```
fechas = ( df_model["date"] .drop_duplicates() .sort_values() .reset_index(drop=True) )
```

Últimas 28 fechas para test final

```
fechas_test = fechas.iloc[-HORIZONTE:] fechas_desarrollo = fechas.iloc[:-HORIZONTE]
```

Separación del dataset

```
df_dev = df_model[ df_model["date"].isin(fechas_desarrollo) ].copy()
```

```
df_test = df_model[ df_model["date"].isin(fechas_test) ].copy()
```

```
print("PARTICIÓN TEMPORAL")
```

```
print("\nDESARROLLO") print("Desde:", df_dev["date"].min()) print("Hasta:",  
df_dev["date"].max()) print("Fechas:", df_dev["date"].nunique()) print("Filas:",  
len(df_dev)) print("Barrios:", df_dev["neighbourhood_cleansed"].nunique())
```

```
print("\nTEST FINAL") print("Desde:", df_test["date"].min()) print("Hasta:",  
df_test["date"].max()) print("Fechas:", df_test["date"].nunique()) print("Filas:",  
len(df_test)) print("Barrios:", df_test["neighbourhood_cleansed"].nunique())
```

```
print("\nVALIDACIONES") print("Fechas compartidas:", len(set(df_dev["date"]) &  
set(df_test["date"])))
```

```
print("Total filas:", len(df_dev) + len(df_test))
```

```
print("Total original:", len(df_model))
```

```
=====
```

BLOQUE DE CÓDIGO 28

```
=====
```

VALIDACIÓN TEMPORAL - 3 FOLDS DE 28 DÍAS

N_FOLDS = 3 VENTANA_VALIDACION = 28

Fechas disponibles únicamente en desarrollo

```
fechas_dev = ( df_dev["date"] .drop_duplicates() .sort_values() .reset_index(drop=True) )
```

```
folds_temporales = []
```

```
for fold in range(N_FOLDS):
```

```
# Las ventanas de validación se construyen desde el final  
# del período de desarrollo hacia atrás
```

```
fin_val = len(fechas_dev) - fold * VENTANA_VALIDACION
```

```
inicio_val = fin_val - VENTANA_VALIDACION
```

```
fechas_train = fechas_dev.iloc[:inicio_val]
```

```
fechas_val = fechas_dev.iloc[inicio_val:fin_val]
```

```

folds_temporales.append({
    "fold": N_FOLDS - fold,
    "train_inicio": fechas_train.min(),
    "train_fin": fechas_train.max(),
    "train_dias": len(fechas_train),
    "val_inicio": fechas_val.min(),
    "val_fin": fechas_val.max(),
    "val_dias": len(fechas_val)
})

```

Orden cronológico de los folds

```

folds_temporales = sorted( folds_temporales, key=lambda x: x["val_inicio"] )

```

```

print(" VALIDACIÓN TEMPORAL \n")

```

```

for i, f in enumerate(folds_temporales, start=1):

```

```

    print(f"FOLD {i}")

```

```

    print(
        f"Entrenamiento: {f['train_inicio'].date()} "
        f"→ {f['train_fin'].date()} "
        f"({f['train_dias']} días)"
    )

```

```

    print(
        f"Validación: {f['val_inicio'].date()} "
        f"→ {f['val_fin'].date()} "
        f"({f['val_dias']} días)"
    )

```

```

    print()

```

```

=====

```

BLOQUE DE CÓDIGO 29

```

=====

```

CREACIÓN DEL LAG SEMANAL (7 DÍAS)

Ordenar correctamente antes de crear el lag

```

df_features = ( df_model .sort_values(["neighbourhood_cleansed", "date"]) .copy() )

```

Ocupación estimada del mismo barrio 7 días antes

```

df_features["lag_7"] = ( df_features .groupby("neighbourhood_cleansed") ["occupancy_rate"]
    .shift(7) )

```

```

print("VALIDACIÓN LAG_7 ")

```

```

print("Filas totales:", len(df_features)) print("Nulos en lag_7:",
df_features["lag_7"].isna().sum())

```

```

print("\nPrimeras observaciones:") print( df_features[ [ "date", "neighbourhood_cleansed",
"occupancy_rate", "lag_7" ] ].head(12) )

```

```

=====

```

BLOQUE DE CÓDIGO 30

```

=====

BASELINE ESTACIONAL SEMANAL - LAG 7 Ocupación 7 días atrás

Evaluación en los 3 folds temporales

resultados_baseline = []

for i, f in enumerate(folds_temporales, start=1):

# Seleccionar únicamente el período de validación
val = df_features[
    (df_features["date"] >= f["val_inicio"]) &
    (df_features["date"] <= f["val_fin"])
].copy()

# El baseline predice el valor observado 7 días antes
y_real = val["occupancy_rate"]
y_pred = val["lag_7"]

mae = mean_absolute_error(y_real, y_pred)

resultados_baseline.append({
    "Fold": i,
    "Inicio validación": f["val_inicio"],
    "Fin validación": f["val_fin"],
    "Observaciones": len(val),
    "MAE": mae
})

resultados_baseline = pd.DataFrame(resultados_baseline)

print(" BASELINE ESTACIONAL - LAG 7 ") print(resultados_baseline.to_string(index=False))

print( "\nMAE promedio:", resultados_baseline["MAE"].mean() )

=====

BLOQUE DE CÓDIGO 31

=====

MODELO TRIVIAL - MEDIA DEL ENTRENAMIENTO

Evaluación en los mismos 3 folds temporales

resultados_trivial = []

for i, f in enumerate(folds_temporales, start=1):

# Datos de entrenamiento del fold
train = df_features[
    (df_features["date"] >= f["train_inicio"]) &
    (df_features["date"] <= f["train_fin"])
].copy()

# Datos de validación del fold
val = df_features[
    (df_features["date"] >= f["val_inicio"]) &
    (df_features["date"] <= f["val_fin"])
].copy()

# Predicción trivial:
# siempre predecir la media observada en entrenamiento

```

```

media_train = train["occupancy_rate"].mean()

y_real = val["occupancy_rate"]
y_pred = [media_train] * len(val)

mae = mean_absolute_error(y_real, y_pred)

resultados_trivial.append({
    "Fold": i,
    "Media entrenamiento": media_train,
    "Observaciones": len(val),
    "MAE": mae
})

resultados_trivial = pd.DataFrame(resultados_trivial)

print(" MODELO TRIVIAL - MEDIA ") print(resultados_trivial.to_string(index=False))

print( "\nMAE promedio:", resultados_trivial["MAE"].mean() )

=====

BLOQUE DE CÓDIGO 32

=====

INGENIERÍA DE VARIABLES PARA MODELADO

Asegurar orden cronológico por barrio

df_features = ( df_model .sort_values(["neighbourhood_cleansed", "date"]) .copy() )

LAGS TEMPORALES

df_features["lag_7"] = ( df_features .groupby("neighbourhood_cleansed") ["occupancy_rate"]
    .shift(7) )

df_features["lag_14"] = ( df_features .groupby("neighbourhood_cleansed") ["occupancy_rate"]
    .shift(14) )

df_features["lag_28"] = ( df_features .groupby("neighbourhood_cleansed") ["occupancy_rate"]
    .shift(28) )

VARIABLES DE CALENDARIO

df_features["day_of_week"] = df_features["date"].dt.dayofweek df_features["month"] =
df_features["date"].dt.month

print("FEATURES CREADAS ")

print( df_features[ [ "date", "neighbourhood_cleansed", "occupancy_rate", "lag_7",
    "lag_14", "lag_28", "day_of_week", "month" ] ].head(35) )

print("\nNULOS GENERADOS ")

print( df_features[ [ "lag_7", "lag_14", "lag_28" ] ].isna().sum() )

=====

BLOQUE DE CÓDIGO 33

=====

```

DATASET LISTO PARA MODELADO

```
features = [ "lag_7", "lag_14", "lag_28", "day_of_week", "month", "neighbourhood_cleansed" ]
```

```
target = "occupancy_rate"
```

Eliminar las primeras observaciones de cada barrio

que todavía no tienen suficiente historia para lag_28

```
df_ml = df_features.dropna( subset=["lag_7", "lag_14", "lag_28"] ).copy()
```

```
print("Dataset con features:", df_features.shape) print("Dataset para modelado:",  
df_ml.shape) print("Filas eliminadas:", len(df_features) - len(df_ml))
```

```
print("\nPeriodo:") print(df_ml["date"].min(), "→", df_ml["date"].max())
```

```
print("\nFechas:", df_ml["date"].nunique()) print("Barrios:",  
df_ml["neighbourhood_cleansed"].nunique())
```

```
print("\nNulos en variables del modelo:") print(df_ml[features + [target]].isnull().sum())
```

```
print("\nDuplicados fecha-barrio:") print( df_ml.duplicated( subset=["date",  
"neighbourhood_cleansed"] ).sum() )
```

```
=====
```

BLOQUE DE CÓDIGO 34

```
=====
```

DEFINICIÓN DE VARIABLES DEL MODELO

```
NUM = [ "lag_7", "lag_14", "lag_28", "day_of_week", "month" ]
```

```
CAT = [ "neighbourhood_cleansed" ]
```

```
TARGET = "occupancy_rate"
```

Variables predictoras

```
X = df_ml[NUM + CAT].copy()
```

Variable objetivo

```
y = df_ml[TARGET].copy()
```

```
=====
```

BLOQUE DE CÓDIGO 35

```
=====
```

PARTICIÓN TEMPORAL: DESARROLLO Y TEST FINAL

```
FECHA_TEST_INICIO = pd.Timestamp("2027-05-27")
```

Máscaras temporales

```
mask_dev = df_ml["date"] < FECHA_TEST_INICIO mask_test = df_ml["date"] >= FECHA_TEST_INICIO
```

Desarrollo

```

X_dev = X.loc[mask_dev].copy() y_dev = y.loc[mask_dev].copy()

Test final

X_test = X.loc[mask_test].copy() y_test = y.loc[mask_test].copy()

Fechas para control

fechas_dev = df_ml.loc[mask_dev, "date"] fechas_test = df_ml.loc[mask_test, "date"]

print("DESARROLLO") print("X:", X_dev.shape) print("y:", y_dev.shape) print( "Periodo:",
fechas_dev.min().date(), "→", fechas_dev.max().date() ) print("Fechas:",
fechas_dev.nunique())

print("\nTEST FINAL") print("X:", X_test.shape) print("y:", y_test.shape) print(
"Periodo:", fechas_test.min().date(), "→", fechas_test.max().date() ) print("Fechas:",
fechas_test.nunique())

print("\nControl") print("Total filas:", len(X_dev) + len(X_test)) print("Total esperado:",
len(df_ml)) print( "Fechas compartidas:", len( set(fechas_dev.unique()) &
set(fechas_test.unique()) ) )

```

=====

BLOQUE DE CÓDIGO 36

=====

PIPELINE DEL MODELO XGBOOST

1. Preprocesamiento

```

preprocessor_xgb = ColumnTransformer( transformers=[ ("num", "passthrough", NUM), ("cat",
OneHotEncoder( handle_unknown="ignore", sparse_output=False ), CAT) ] )

```

2. Modelo XGBoost inicial

```

xgb_model = XGBRegressor( objective="reg:squarederror", random_state=42, n_jobs=-1 )

```

3. Pipeline completo

```

pipeline_xgb = Pipeline([ ("preprocessor", preprocessor_xgb), ("model", xgb_model) ])
print(pipeline_xgb)

```

=====

BLOQUE DE CÓDIGO 37

=====

FOLDS TEMPORALES PARA VALIDACIÓN

Fechas únicas disponibles en desarrollo

```

fechas_dev_unicas = ( df_ml.loc[mask_dev, "date"] .drop_duplicates() .sort_values()
.reset_index(drop=True) )

```

N_FOLDS = 3 HORIZONTE = 28

folds_temporales = []

for fold in range(N_FOLDS):

```

val_end = len(fechas_dev_unicas) - (N_FOLDS - 1 - fold) * HORIZONTE
val_start = val_end - HORIZONTE

fechas_train_fold = fechas_dev_unicas.iloc[:val_start]
fechas_val_fold = fechas_dev_unicas.iloc[val_start:val_end]

folds_temporales.append(
    (fechas_train_fold, fechas_val_fold)
)

Mostrar estructura

for i, (fechas_train_fold, fechas_val_fold) in enumerate( folds_temporales, start=1 ):
    print(f"\nFOLD {i}")

    print(
        "Train:",
        fechas_train_fold.min().date(),
        "→",
        fechas_train_fold.max().date(),
        f"({len(fechas_train_fold)} días)"
    )

    print(
        "Validación:",
        fechas_val_fold.min().date(),
        "→",
        fechas_val_fold.max().date(),
        f"({len(fechas_val_fold)} días)"
    )

    print(
        "Fechas compartidas:",
        len(
            set(fechas_train_fold)
            & set(fechas_val_fold)
        )
    )

=====

BLOQUE DE CÓDIGO 38

=====

XGBOOST BASE - VALIDACIÓN TEMPORAL

resultados_xgb_base = []

for i, (fechas_train_fold, fechas_val_fold) in enumerate( folds_temporales, start=1 ):

    # -----
    # Máscaras según las fechas de cada fold
    # -----

    mask_train_fold = df_ml["date"].isin(fechas_train_fold)
    mask_val_fold = df_ml["date"].isin(fechas_val_fold)

    # -----
    # Datos de entrenamiento
    # -----

    X_train_fold = X.loc[mask_train_fold]

```

```

y_train_fold = y.loc[mask_train_fold]

# -----
# Datos de validación
# -----

X_val_fold = X.loc[mask_val_fold]
y_val_fold = y.loc[mask_val_fold]

# -----
# Modelo nuevo para este fold
# -----

modelo_fold = clone(pipeline_xgb)

# Entrenamiento
modelo_fold.fit(
    X_train_fold,
    y_train_fold
)

# Predicción
y_pred_fold = modelo_fold.predict(X_val_fold)

# MAE
mae_fold = mean_absolute_error(
    y_val_fold,
    y_pred_fold
)

resultados_xgb_base.append(mae_fold)

print(f"\nFOLD {i}")
print("Train:", X_train_fold.shape)
print("Validación:", X_val_fold.shape)
print(f"MAE: {mae_fold:.6f}")

Resultado promedio

mae_xgb_base = np.mean(resultados_xgb_base)

print("\nXGBOOST BASE") print("=====")

print("MAE por fold:") for i, mae in enumerate(resultados_xgb_base, start=1): print(f"Fold
{i}: {mae:.6f}")

print(f"\nMAE promedio: {mae_xgb_base:.6f}")

=====

BLOQUE DE CÓDIGO 39

=====

PRESUPUESTO DE OPTIMIZACIÓN - XGBOOST

10 configuraciones

configuraciones_xgb = [

{"n_estimators": 100, "max_depth": 3, "learning_rate": 0.05, "subsample": 0.8},
{"n_estimators": 200, "max_depth": 3, "learning_rate": 0.05, "subsample": 0.8},
{"n_estimators": 300, "max_depth": 3, "learning_rate": 0.05, "subsample": 0.8},

```

```

{"n_estimators": 100, "max_depth": 5, "learning_rate": 0.05, "subsample": 0.8},
{"n_estimators": 200, "max_depth": 5, "learning_rate": 0.05, "subsample": 0.8},

{"n_estimators": 300, "max_depth": 5, "learning_rate": 0.03, "subsample": 0.8},

{"n_estimators": 100, "max_depth": 7, "learning_rate": 0.05, "subsample": 0.8},
{"n_estimators": 200, "max_depth": 7, "learning_rate": 0.03, "subsample": 0.8},

{"n_estimators": 300, "max_depth": 7, "learning_rate": 0.03, "subsample": 0.9},

{"n_estimators": 200, "max_depth": 5, "learning_rate": 0.10, "subsample": 0.9}

]

```

```

print("Configuraciones XGBoost:", len(configuraciones_xgb))

```

```

for i, config in enumerate(configuraciones_xgb, start=1): print(f"{i}: {config}")

```

```

=====

BLOQUE DE CÓDIGO 40

=====

```

```

OPTIMIZACIÓN XGBOOST - 10 CONFIGURACIONES × 3 FOLDS

```

```

resultados_opt_xgb = []

```

```

for num_config, config in enumerate(configuraciones_xgb, start=1):

```

```

    maes_folds = []

```

```

    print(f"\nCONFIGURACIÓN {num_config}/10")

```

```

    for num_fold, (fechas_train_fold, fechas_val_fold) in enumerate(
        folds_temporales, start=1
    ):

```

```

        # Máscaras temporales

```

```

        mask_train_fold = df_ml["date"].isin(fechas_train_fold)
        mask_val_fold = df_ml["date"].isin(fechas_val_fold)

```

```

        # Datos del fold

```

```

        X_train_fold = X.loc[mask_train_fold]
        y_train_fold = y.loc[mask_train_fold]

```

```

        X_val_fold = X.loc[mask_val_fold]
        y_val_fold = y.loc[mask_val_fold]

```

```

        # Pipeline limpio

```

```

        modelo_fold = clone(pipeline_xgb)

```

```

        # Aplicar hiperparámetros de esta configuración

```

```

        modelo_fold.set_params(
            model__n_estimators=config["n_estimators"],
            model__max_depth=config["max_depth"],
            model__learning_rate=config["learning_rate"],
            model__subsample=config["subsample"]
        )

```

```

        # Entrenar

```

```

modelo_fold.fit(
    X_train_fold,
    y_train_fold
)

# Predecir
y_pred_fold = modelo_fold.predict(X_val_fold)

# MAE
mae_fold = mean_absolute_error(
    y_val_fold,
    y_pred_fold
)

maes_folds.append(mae_fold)

print(
    f"Fold {num_fold}: "
    f"MAE = {mae_fold:.6f}"
)

# Resultado de la configuración
resultados_opt_xgb.append({
    "configuracion": num_config,
    "n_estimators": config["n_estimators"],
    "max_depth": config["max_depth"],
    "learning_rate": config["learning_rate"],
    "subsample": config["subsample"],
    "mae_fold_1": maes_folds[0],
    "mae_fold_2": maes_folds[1],
    "mae_fold_3": maes_folds[2],
    "mae_promedio": np.mean(maes_folds)
})

```

TABLA DE RESULTADOS

```

resultados_opt_xgb = pd.DataFrame(resultados_opt_xgb)

resultados_opt_xgb = resultados_opt_xgb.sort_values( "mae_promedio"
).reset_index(drop=True)

print("\n=====") print("RANKING XGBOOST")
print("=====")

display( resultados_opt_xgb.round(6) )

```

BLOQUE DE CÓDIGO 41

PREPROCESAMIENTO PARA MLP

```

preprocessor_mlp = ColumnTransformer( transformers=[ ( "num", StandardScaler(), NUM ), (
"cat", OneHotEncoder( handle_unknown="ignore", sparse_output=False ), CAT ) ] )

print(preprocessor_mlp)

```

BLOQUE DE CÓDIGO 42

FUNCIÓN PARA CREAR LA MLP BASE

```
def crear_mlp_base(input_dim):
```

```
    set_random_seed(SEED)
```

```
    model = Sequential([
        Input(shape=(input_dim,)),
```

```
        Dense(64, activation="relu"),
        Dropout(0.20),
```

```
        Dense(32, activation="relu"),
        Dropout(0.20),
```

```
        Dense(1)
```

```
    ])
```

```
    model.compile(
        optimizer=Adam(),
        loss="mae",
        metrics=["mae"]
    )
```

```
    return model
```

=====

BLOQUE DE CÓDIGO 43

=====

MLP BASE - VALIDACIÓN TEMPORAL

```
resultados_mlp_base = [] historias_mlp_base = []
```

```
for i, (fechas_train_fold, fechas_val_fold) in enumerate( folds_temporales, start=1 ):
```

```
    # Reproducibilidad
    set_random_seed(42)
```

```
    # Máscaras temporales
```

```
    mask_train_fold = df_ml["date"].isin(fechas_train_fold)
    mask_val_fold = df_ml["date"].isin(fechas_val_fold)
```

```
    # Datos del fold
```

```
    X_train_fold = X.loc[mask_train_fold]
    y_train_fold = y.loc[mask_train_fold]
```

```
    X_val_fold = X.loc[mask_val_fold]
    y_val_fold = y.loc[mask_val_fold]
```

```
    # Preprocesamiento NUEVO para cada fold
```

```
    preprocessor_fold = clone(preprocessor_mlp)
```

```
    X_train_mlp = preprocessor_fold.fit_transform(
```

```

        X_train_fold
    )

X_val_mlp = preprocessor_fold.transform(
    X_val_fold
)

# Crear una MLP NUEVA para cada fold

modelo_mlp = crear_mlp_base(
    input_dim=X_train_mlp.shape[1]
)

# Early Stopping

early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=15,
    restore_best_weights=True
)

# Entrenamiento

historia = modelo_mlp.fit(
    X_train_mlp,
    y_train_fold,
    validation_data=(
        X_val_mlp,
        y_val_fold
    ),
    epochs=200,
    batch_size=64,
    callbacks=[early_stopping],
    verbose=0
)

# Predicción

y_pred_fold = modelo_mlp.predict(
    X_val_mlp,
    verbose=0
).ravel()

# MAE

mae_fold = mean_absolute_error(
    y_val_fold,
    y_pred_fold
)

resultados_mlp_base.append(mae_fold)
historias_mlp_base.append(historia.history)

mejor_epoca = (
    np.argmin(historia.history["val_loss"]) + 1

```

```

)

print(f"\nFOLD {i}")
print("Train:", X_train_fold.shape)
print("Validación:", X_val_fold.shape)
print("Entradas MLP:", X_train_mlp.shape[1])
print("Épocas ejecutadas:", len(historia.history["loss"]))
print("Mejor época:", mejor_epoca)
print(f"MAE: {mae_fold:.6f}")

RESULTADO PROMEDIO

=====

mae_mlp_base = np.mean(resultados_mlp_base)

print("\nMLP BASE") print("=====")

print("MAE por fold:")

for i, mae in enumerate(resultados_mlp_base, start=1): print(f"Fold {i}: {mae:.6f}")

print(f"\nMAE promedio: {mae_mlp_base:.6f}")

=====

BLOQUE DE CÓDIGO 44

=====

PRESUPUESTO DE OPTIMIZACIÓN - MLP

10 configuraciones

configuraciones_mlp = [

{"capas": (32,),      "dropout": 0.10, "learning_rate": 0.001, "batch_size": 64},
{"capas": (64,),      "dropout": 0.10, "learning_rate": 0.001, "batch_size": 64},

{"capas": (64, 32),   "dropout": 0.10, "learning_rate": 0.001, "batch_size": 64},
{"capas": (64, 32),   "dropout": 0.20, "learning_rate": 0.0005, "batch_size": 64},
{"capas": (64, 32),   "dropout": 0.30, "learning_rate": 0.001, "batch_size": 64},

{"capas": (128, 64),  "dropout": 0.20, "learning_rate": 0.001, "batch_size": 64},
{"capas": (128, 64),  "dropout": 0.30, "learning_rate": 0.0005, "batch_size": 64},

{"capas": (128, 64, 32), "dropout": 0.20, "learning_rate": 0.001, "batch_size": 64},
{"capas": (128, 64, 32), "dropout": 0.30, "learning_rate": 0.0005, "batch_size": 64},

{"capas": (64, 32),   "dropout": 0.20, "learning_rate": 0.001, "batch_size": 32}

]

print("Configuraciones MLP:", len(configuraciones_mlp))

for i, config in enumerate(configuraciones_mlp, start=1): print(f"{i}: {config}")

=====

```

BLOQUE DE CÓDIGO 45

=====

FUNCIÓN PARA CREAR MLP CONFIGURABLE

```
def crear_mlp(input_dim, capas, dropout, learning_rate):  
  
    set_random_seed(SEED)  
  
    model = Sequential()  
  
    model.add(Input(shape=(input_dim,)))  
  
    for neuronas in capas:  
        model.add(  
            Dense(  
                neuronas,  
                activation="relu"  
            )  
        )  
  
        model.add(  
            Dropout(dropout)  
        )  
  
    model.add(Dense(1))  
  
    model.compile(  
        optimizer=Adam(  
            learning_rate=learning_rate  
        ),  
        loss="mae",  
        metrics=["mae"]  
    )  
  
    return model
```

=====

BLOQUE DE CÓDIGO 46

=====

OPTIMIZACIÓN MLP - 10 CONFIGURACIONES × 3 FOLDS

```
resultados_opt_mlp = []  
  
for num_config, config in enumerate(configuraciones_mlp, start=1):  
  
    maes_folds = []  
    mejores_epocas = []  
  
    print(f"\nCONFIGURACIÓN {num_config}/10")  
  
    for num_fold, (fechas_train_fold, fechas_val_fold) in enumerate(  
        folds_temporales, start=1  
    ):  
  
        # Reproducibilidad  
        set_random_seed(42)  
  
        # Máscaras temporales  
        mask_train_fold = df_ml["date"].isin(fechas_train_fold)  
        mask_val_fold = df_ml["date"].isin(fechas_val_fold)
```

```

# Datos del fold
X_train_fold = X.loc[mask_train_fold]
y_train_fold = y.loc[mask_train_fold]

X_val_fold = X.loc[mask_val_fold]
y_val_fold = y.loc[mask_val_fold]

# Preprocesamiento NUEVO para cada fold
preprocessor_fold = clone(preprocessor_mlp)

X_train_mlp = preprocessor_fold.fit_transform(
    X_train_fold
)

X_val_mlp = preprocessor_fold.transform(
    X_val_fold
)

# Crear MLP según configuración
modelo_mlp = crear_mlp(
    input_dim=X_train_mlp.shape[1],
    capas=config["capas"],
    dropout=config["dropout"],
    learning_rate=config["learning_rate"]
)

# Early Stopping
early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=15,
    restore_best_weights=True
)

# Entrenamiento
historia = modelo_mlp.fit(
    X_train_mlp,
    y_train_fold,
    validation_data=(
        X_val_mlp,
        y_val_fold
    ),
    epochs=200,
    batch_size=config["batch_size"],
    callbacks=[early_stopping],
    verbose=0
)

# Predicción
y_pred_fold = modelo_mlp.predict(
    X_val_mlp,
    verbose=0
).ravel()

# MAE

```

```

mae_fold = mean_absolute_error(
    y_val_fold,
    y_pred_fold
)

mejor_epoca = (
    np.argmin(historia.history["val_loss"]) + 1
)

maes_folds.append(mae_fold)
mejores_epocas.append(mejor_epoca)

print(
    f"Fold {num_fold}: "
    f"MAE = {mae_fold:.6f} | "
    f"Mejor época = {mejor_epoca}"
)

# Guardar resultado de esta configuración

resultados_opt_mlp.append({

    "configuracion": num_config,

    "capas": str(config["capas"]),
    "dropout": config["dropout"],
    "learning_rate": config["learning_rate"],
    "batch_size": config["batch_size"],

    "mae_fold_1": maes_folds[0],
    "mae_fold_2": maes_folds[1],
    "mae_fold_3": maes_folds[2],

    "mae_promedio": np.mean(maes_folds),

    "mejor_epoca_1": mejores_epocas[0],
    "mejor_epoca_2": mejores_epocas[1],
    "mejor_epoca_3": mejores_epocas[2]
})

```

TABLA DE RESULTADOS

```

=====

resultados_opt_mlp = pd.DataFrame( resultados_opt_mlp )

resultados_opt_mlp = resultados_opt_mlp.sort_values( "mae_promedio"
).reset_index(drop=True)

print("\nRANKING MLP") print("=====")

display( resultados_opt_mlp.round(6) )

=====

```

BLOQUE DE CÓDIGO 47

COMPARACIÓN: SPLIT ALEATORIO (INGENUO)

Split aleatorio SOLO dentro del conjunto de desarrollo

```

X_train_random, X_val_random, y_train_random, y_val_random = ( train_test_split( X_dev,
y_dev, test_size=28/300, random_state=42 ) )

XGBoost optimizado

xgb_random = clone(pipeline_xgb)

xgb_random.set_params( model__n_estimators=300, model__max_depth=3,
model__learning_rate=0.05, model__subsample=0.8 )

Entrenar

xgb_random.fit( X_train_random, y_train_random )

Predecir

y_pred_random = xgb_random.predict( X_val_random )

MAE

mae_random = mean_absolute_error( y_val_random, y_pred_random )

print("SPLIT ALEATORIO") print("-----") print("Train:",
X_train_random.shape) print("Validación:", X_val_random.shape) print(f"MAE:
{mae_random:.6f}")

print("\nVALIDACIÓN TEMPORAL") print("-----") print(f"MAE
promedio: {0.021687:.6f}")

=====

BLOQUE DE CÓDIGO 48

=====

#CURVA DE APRENDIZAJE TEMPORAL - XGBOOST OPTIMIZADO

Fold 3 como referencia

fechas_train_lc, fechas_val_lc = folds_temporales[2]

fracciones = [0.10, 0.25, 0.50, 0.75, 1.00]

resultados_learning_curve = []

for fraccion in fracciones:

# Cantidad de fechas de entrenamiento
n_fechas = max(
    1,
    int(len(fechas_train_lc) * fraccion)
)

# Utilizamos las fechas MÁS RECIENTES del entrenamiento
# para mantenerlas inmediatamente antes de la validación
fechas_train_fraccion = fechas_train_lc.iloc[-n_fechas:]

# Máscaras
mask_train_lc = df_ml["date"].isin(
    fechas_train_fraccion
)

mask_val_lc = df_ml["date"].isin(

```

```

        fechas_val_lc
    )

# Datos
X_train_lc = X.loc[mask_train_lc]
y_train_lc = y.loc[mask_train_lc]

X_val_lc = X.loc[mask_val_lc]
y_val_lc = y.loc[mask_val_lc]

# XGBoost optimizado
modelo_lc = clone(pipeline_xgb)

modelo_lc.set_params(
    model__n_estimators=300,
    model__max_depth=3,
    model__learning_rate=0.05,
    model__subsample=0.8
)

# Entrenamiento
modelo_lc.fit(
    X_train_lc,
    y_train_lc
)

# Predicción
y_pred_lc = modelo_lc.predict(
    X_val_lc
)

# MAE
mae_lc = mean_absolute_error(
    y_val_lc,
    y_pred_lc
)

resultados_learning_curve.append({
    "porcentaje": int(fraccion * 100),
    "dias_train": len(fechas_train_fraccion),
    "filas_train": len(X_train_lc),
    "fecha_inicio": fechas_train_fraccion.min().date(),
    "fecha_fin": fechas_train_fraccion.max().date(),
    "mae_validacion": mae_lc
})

Resultados

resultados_learning_curve = pd.DataFrame( resultados_learning_curve )

display( resultados_learning_curve.round(6) )

=====

```

BLOQUE DE CÓDIGO 49

=====

Configuración ganadora ya seleccionada

```

mejor_config_mlp = { "capas": (64,), "dropout": 0.10, "learning_rate": 0.001, "batch_size":
64 }

historias_mlp_opt = [] resultados_curvas_mlp_opt = []

for i, (fechas_train, fechas_val) in enumerate(folds_temporales, start=1):

print(f"\nEntrenando Fold {i}...")

# Máscaras basadas en las fechas de cada fold
mask_train_fold = fechas_dev.isin(fechas_train)
mask_val_fold = fechas_dev.isin(fechas_val)

X_train_fold = X_dev.loc[mask_train_fold]
y_train_fold = y_dev.loc[mask_train_fold]

X_val_fold = X_dev.loc[mask_val_fold]
y_val_fold = y_dev.loc[mask_val_fold]

print(
    f"Train: {len(X_train_fold):,} filas | "
    f"Validación: {len(X_val_fold):,} filas"
)

# Preprocesamiento ajustado SOLO con entrenamiento
pre_fold = clone(preprocessor_mlp)

X_train_proc = pre_fold.fit_transform(X_train_fold)
X_val_proc = pre_fold.transform(X_val_fold)

# MLP optimizado
modelo = crear_mlp(
    input_dim=X_train_proc.shape[1],
    capas=mejor_config_mlp["capas"],
    dropout=mejor_config_mlp["dropout"],
    learning_rate=mejor_config_mlp["learning_rate"]
)

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=15,
    restore_best_weights=True
)

historia = modelo.fit(
    X_train_proc,
    y_train_fold,
    validation_data=(X_val_proc, y_val_fold),
    epochs=200,
    batch_size=mejor_config_mlp["batch_size"],
    callbacks=[early_stop],
    verbose=0
)

# Guardamos curvas
historias_mlp_opt.append(historia.history)

val_loss = historia.history["val_loss"]

mejor_epoca = int(np.argmin(val_loss)) + 1

resultados_curvas_mlp_opt.append({
    "fold": i,
    "epocas_ejecutadas": len(val_loss),

```

```

        "mejor_epoca": mejor_epoca,
        "mejor_val_loss": min(val_loss)
    })

print(
    f"Fold {i}: "
    f"{len(val_loss)} épocas ejecutadas | "
    f"Mejor época = {mejor_epoca} | "
    f"Mejor val_loss = {min(val_loss):.6f}"
)

```

=====

BLOQUE DE CÓDIGO 50

=====

```

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for i, historia in enumerate(historias_mlp_opt):

    loss = historia["loss"]
    val_loss = historia["val_loss"]

    mejor_epoca = int(np.argmin(val_loss)) + 1

    axes[i].plot(
        range(1, len(loss) + 1),
        loss,
        label="Entrenamiento"
    )

    axes[i].plot(
        range(1, len(val_loss) + 1),
        val_loss,
        label="Validación"
    )

    axes[i].axvline(
        mejor_epoca,
        linestyle="--",
        label=f"Mejor época: {mejor_epoca}"
    )

    axes[i].set_title(f"Fold {i + 1}")
    axes[i].set_xlabel("Época")
    axes[i].set_ylabel("MAE")
    axes[i].legend()
    axes[i].grid(alpha=0.3)

plt.suptitle( "Curvas de pérdida del MLP Optimizado - Validación temporal", fontsize=15 )

plt.tight_layout() plt.show()

```

=====

BLOQUE DE CÓDIGO 51

=====

```

Fold 3

fechas_train_xgb, fechas_val_xgb = folds_temporales[2]

```

```

mask_train_xgb = fechas_dev.isin(fechas_train_xgb) mask_val_xgb =
fechas_dev.isin(fechas_val_xgb)

X_train_xgb = X_dev.loc[mask_train_xgb] y_train_xgb = y_dev.loc[mask_train_xgb]

X_val_xgb = X_dev.loc[mask_val_xgb] y_val_xgb = y_dev.loc[mask_val_xgb]

Modelo XGBoost con la configuración ganadora

modelo_xgb_interpret = XGBRegressor( objective="reg:squarederror", n_estimators=300,
max_depth=3, learning_rate=0.05, subsample=0.8, random_state=42, n_jobs=-1 )

Pipeline completo

pipeline_xgb_interpret = Pipeline([ ("preprocessor", clone(preprocessor_xgb)), ("model",
modelo_xgb_interpret) ])

Entrenamiento únicamente con el Fold 3

pipeline_xgb_interpret.fit( X_train_xgb, y_train_xgb )

print("XGBoost preparado para interpretabilidad") print(f"Train: {len(X_train_xgb):,}
filas") print(f"Validación: {len(X_val_xgb):,} filas")

print( "Train:", fechas_train_xgb.min().date(), "→", fechas_train_xgb.max().date() )

print( "Validación:", fechas_val_xgb.min().date(), "→", fechas_val_xgb.max().date() )

```

BLOQUE DE CÓDIGO 52

```

=====

perm_xgb = permutation_importance( pipeline_xgb_interpret, X_val_xgb, y_val_xgb,
scoring="neg_mean_absolute_error", n_repeats=10, random_state=42, n_jobs=1 )

importancia_xgb = pd.DataFrame({ "Variable": X_val_xgb.columns, "Importancia_MAE":
perm_xgb.importances_mean, "Desviacion": perm_xgb.importances_std }).sort_values(
"Importancia_MAE", ascending=False ).reset_index(drop=True)

display(importancia_xgb)

```

BLOQUE DE CÓDIGO 53

```

=====

Orden para gráfico horizontal

grafico_xgb = importancia_xgb.sort_values( "Importancia_MAE", ascending=True )

plt.figure(figsize=(9, 5))

plt.barh( grafico_xgb["Variable"], grafico_xgb["Importancia_MAE"],
xerr=grafico_xgb["Desviacion"] )

plt.xlabel("Aumento del MAE al permutar la variable") plt.ylabel("Variable")

plt.title( "Permutation Importance - XGBoost Optimizado\n" "Fold 3 de validación temporal"
)

plt.grid(axis="x", alpha=0.3)

```

```
plt.tight_layout() plt.show()
```

```
=====
```

BLOQUE DE CÓDIGO 54

```
=====
```

Mismo Fold 3 utilizado para XGBoost

```
fechas_train_mlp, fechas_val_mlp = folds_temporales[2]
```

```
mask_train_mlp = fechas_dev.isin(fechas_train_mlp) mask_val_mlp =  
fechas_dev.isin(fechas_val_mlp)
```

```
X_train_mlp = X_dev.loc[mask_train_mlp] y_train_mlp = y_dev.loc[mask_train_mlp]
```

```
X_val_mlp = X_dev.loc[mask_val_mlp] y_val_mlp = y_dev.loc[mask_val_mlp]
```

Preprocesamiento

```
preprocessor_mlp_interpret = clone(preprocessor_mlp)
```

```
X_train_mlp_proc = preprocessor_mlp_interpret.fit_transform(X_train_mlp) X_val_mlp_proc =  
preprocessor_mlp_interpret.transform(X_val_mlp)
```

MLP optimizado

```
modelo_mlp_interpret = crear_mlp( input_dim=X_train_mlp_proc.shape[1], capas=(64,),  
dropout=0.10, learning_rate=0.001 )
```

```
early_stop_interpret = EarlyStopping( monitor="val_loss", patience=15,  
restore_best_weights=True )
```

```
historia_mlp_interpret = modelo_mlp_interpret.fit( X_train_mlp_proc, y_train_mlp,  
validation_data=(X_val_mlp_proc, y_val_mlp), epochs=200, batch_size=64,  
callbacks=[early_stop_interpret], verbose=0 )
```

```
print("MLP preparado para interpretabilidad") print(f"Train: {len(X_train_mlp):,} filas")  
print(f"Validación: {len(X_val_mlp):,} filas") print(f"Variables originales:  
{X_val_mlp.shape[1]}") print(f"Variables después del preprocesamiento:  
{X_val_mlp_proc.shape[1]}")
```

```
=====
```

BLOQUE DE CÓDIGO 55

```
=====
```

Predicción normal del MLP

```
pred_base_mlp = modelo_mlp_interpret.predict( X_val_mlp_proc, verbose=0 ).ravel()
```

```
mae_base_mlp = mean_absolute_error( y_val_mlp, pred_base_mlp )
```

```
print(f"MAE base del MLP: {mae_base_mlp:.6f}")
```

Número de repeticiones

```
n_repeats = 10 rng = np.random.RandomState(42)
```

```
resultados_perm_mlp = []
```

```
for variable in X_val_mlp.columns:
```

```

aumentos_mae = []

for repeticion in range(n_repeats):

    # Copia del conjunto de validación original
    X_perm = X_val_mlp.copy()

    # Permutar únicamente la variable analizada
    X_perm[variable] = rng.permutation(
        X_perm[variable].values
    )

    # Aplicar exactamente el mismo preprocesamiento
    X_perm_proc = preprocessor_mlp_interpret.transform(
        X_perm
    )

    # Predicción con el mismo MLP
    pred_perm = modelo_mlp_interpret.predict(
        X_perm_proc,
        verbose=0
    ).ravel()

    # MAE después de desordenar la variable
    mae_perm = mean_absolute_error(
        y_val_mlp,
        pred_perm
    )

    aumentos_mae.append(
        mae_perm - mae_base_mlp
    )

resultados_perm_mlp.append({
    "Variable": variable,
    "Importancia_MAE": np.mean(aumentos_mae),
    "Desviacion": np.std(aumentos_mae)
})

importancia_mlp = pd.DataFrame( resultados_perm_mlp ).sort_values( "Importancia_MAE",
ascending=False ).reset_index(drop=True)

display(importancia_mlp)

=====

```

BLOQUE DE CÓDIGO 56

=====

Comparación de Permutation Importance: XGBoost vs MLP

```

comparacion_importancia = ( importancia_xgb[["Variable", "Importancia_MAE"]]
.rename(columns={"Importancia_MAE": "XGBoost"}) .merge( importancia_mlp[["Variable",
"Importancia_MAE"]] .rename(columns={"Importancia_MAE": "MLP"}), on="Variable" ) )

```

Ordenar por importancia de XGBoost

```

comparacion_importancia = comparacion_importancia.sort_values( "XGBoost", ascending=True )

```

Gráfica

```

ax = comparacion_importancia.set_index("Variable")[ ["XGBoost", "MLP"] ].plot( kind="barh",
figsize=(9, 5) )

ax.set_xlabel("Aumento del MAE al permutar la variable") ax.set_ylabel("Variable")
ax.set_title( "Permutation Importance: XGBoost vs MLP\n" "Fold 3 de validación temporal" )
ax.grid(axis="x", alpha=0.3)

plt.tight_layout() plt.show()

display( comparacion_importancia.sort_values( "XGBoost", ascending=False ) )

```

=====

BLOQUE DE CÓDIGO 57

=====

```

Predicciones XGBoost - Fold 3

pred_xgb_error = pipeline_xgb_interpret.predict(X_val_xgb)

Tabla de análisis de errores

errores_xgb = df_ml.loc[X_val_xgb.index, [ "date", "neighbourhood_cleansed",
"occupancy_rate" ]].copy()

errores_xgb["Prediccion"] = pred_xgb_error

Residuo con signo

errores_xgb["Residuo"] = ( errores_xgb["occupancy_rate"] - errores_xgb["Prediccion"] )

Error absoluto

errores_xgb["Error_Absoluto"] = abs( errores_xgb["Residuo"] )

print("Número de predicciones:", len(errores_xgb)) print(f"MAE:
{errores_xgb['Error_Absoluto'].mean():.6f}") print(f"Residuo medio:
{errores_xgb['Residuo'].mean():.6f}")

print("\n5 casos con mayor error absoluto:")

display( errores_xgb.sort_values( "Error_Absoluto", ascending=False ).head(5) )

```

=====

BLOQUE DE CÓDIGO 58

=====

```

Predicciones MLP - Fold 3

pred_mlp_error = modelo_mlp_interpret.predict( X_val_mlp_proc, verbose=0 ).ravel()

Tabla de análisis de errores

errores_mlp = df_ml.loc[X_val_mlp.index, [ "date", "neighbourhood_cleansed",
"occupancy_rate" ]].copy()

errores_mlp["Prediccion"] = pred_mlp_error

Residuo = Real - Predicción

errores_mlp["Residuo"] = ( errores_mlp["occupancy_rate"] - errores_mlp["Prediccion"] )

```

Error absoluto

```
errores_mlp["Error_Absoluto"] = abs( errores_mlp["Residuo"] )

print("Número de predicciones:", len(errores_mlp)) print(f"MAE:
{errores_mlp['Error_Absoluto'].mean():.6f}") print(f"Residuo medio:
{errores_mlp['Residuo'].mean():.6f}")

print("\n5 casos con mayor error absoluto:")

display( errores_mlp.sort_values( "Error_Absoluto", ascending=False ).head(5) )
```

=====

BLOQUE DE CÓDIGO 59

=====

Agregar distrito a las tablas de errores

```
errores_xgb["Distrito"] = df_ml.loc[ errores_xgb.index, "neighbourhood_group_cleansed" ]
errores_mlp["Distrito"] = df_ml.loc[ errores_mlp.index, "neighbourhood_group_cleansed" ]
```

MAE por distrito - XGBoost

```
subgrupos_xgb = ( errores_xgb .groupby("Distrito") .agg( MAE_XGBoost=("Error_Absoluto",
"mean"), Observaciones=("Error_Absoluto", "size") ) .reset_index() )
```

MAE por distrito - MLP

```
subgrupos_mlp = ( errores_mlp .groupby("Distrito") .agg( MAE_MLP=("Error_Absoluto",
"mean"), Observaciones_MLP=("Error_Absoluto", "size") ) .reset_index() )
```

Combinar resultados

```
comparacion_subgrupos = subgrupos_xgb.merge( subgrupos_mlp, on="Distrito" )

comparacion_subgrupos = comparacion_subgrupos[ ["Distrito", "Observaciones", "MAE_XGBoost",
"MAE_MLP"] ].sort_values( "MAE_XGBoost", ascending=False ).reset_index(drop=True)

display(comparacion_subgrupos)
```

=====

BLOQUE DE CÓDIGO 60

=====

Ordenar por MAE de XGBoost

```
grafico_subgrupos = comparacion_subgrupos.sort_values( "MAE_XGBoost", ascending=True )

ax = grafico_subgrupos.set_index("Distrito")[ ["MAE_XGBoost", "MAE_MLP"] ].plot(
kind="barh", figsize=(10, 6) )

ax.set_xlabel("MAE") ax.set_ylabel("Distrito") ax.set_title( "MAE por distrito: XGBoost vs
MLP\n" "Fold 3 de validación temporal" ) ax.grid(axis="x", alpha=0.3)

plt.tight_layout() plt.show()
```

=====

BLOQUE DE CÓDIGO 61

```

=====

EVALUACIÓN FINAL - VERIFICACIÓN ANTES DE ABRIR EL TEST

print("DESARROLLO") print(f"Filas: {len(X_dev):,}") print(f"Fechas:
{fechas_dev.min().date()} -> {fechas_dev.max().date()}")

print("\nTEST FINAL") print(f"Filas: {len(X_test):,}") print(f"Fechas:
{fechas_test.min().date()} -> {fechas_test.max().date()}") print(f"Días únicos:
{fechas_test.nunique()}")

print("\nCONFIGURACIÓN XGBOOST CONGELADA") print("n_estimators = 300") print("max_depth =
3") print("learning_rate = 0.05") print("subsample = 0.80")

print("\nCONFIGURACIÓN MLP CONGELADA") print("capas = (64,)") print("dropout = 0.10")
print("learning_rate = 0.001") print("batch_size = 64")

print("\nFeatures:") print(NUM + CAT)

=====

BLOQUE DE CÓDIGO 62

=====

ENTRENAMIENTO FINAL - XGBOOST

modelo_xgb_final = XGBRegressor( objective="reg:squarederror", n_estimators=300,
max_depth=3, learning_rate=0.05, subsample=0.80, random_state=42, n_jobs=-1 )

pipeline_xgb_final = Pipeline([ ("preprocessor", clone(preprocessor_xgb)), ("model",
modelo_xgb_final) ])

inicio = time.perf_counter()

pipeline_xgb_final.fit( X_dev, y_dev )

tiempo_train_xgb = time.perf_counter() - inicio

ENTRENAMIENTO FINAL - MLP

preprocessor_mlp_final = clone(preprocessor_mlp)

inicio = time.perf_counter()

X_dev_mlp_proc = preprocessor_mlp_final.fit_transform(X_dev)

modelo_mlp_final = crear_mlp( input_dim=X_dev_mlp_proc.shape[1], capas=(64,), dropout=0.10,
learning_rate=0.001 )

modelo_mlp_final.fit( X_dev_mlp_proc, y_dev, epochs=13, batch_size=64, verbose=0 )

tiempo_train_mlp = time.perf_counter() - inicio

print("ENTRENAMIENTO FINAL COMPLETADO") print(f"XGBoost: {tiempo_train_xgb:.4f} segundos")
print(f"MLP: {tiempo_train_mlp:.4f} segundos")

print("\nDatos utilizados:") print(f"XGBoost: {len(X_dev):,} observaciones") print(f"MLP:
{len(X_dev):,} observaciones")

=====

BLOQUE DE CÓDIGO 63

```

```

=====

TEST FINAL - EVALUACIÓN ÚNICA

1. BASELINE DE LA MEDIA

La media se calcula únicamente con DEVELOPMENT

media_dev = y_dev.mean()

pred_test_media = np.full( shape=len(y_test), fill_value=media_dev )

mae_test_media = mean_absolute_error( y_test, pred_test_media )

2. BASELINE ESTACIONAL LAG 7

pred_test_lag7 = X_test["lag_7"].values

mae_test_lag7 = mean_absolute_error( y_test, pred_test_lag7 )

3. XGBOOST FINAL

Incluye preprocesamiento + predicción

inicio = time.perf_counter()

pred_test_xgb = pipeline_xgb_final.predict( X_test )

tiempo_inferencia_xgb = time.perf_counter() - inicio

mae_test_xgb = mean_absolute_error( y_test, pred_test_xgb )

4. MLP FINAL

Incluye preprocesamiento + predicción

inicio = time.perf_counter()

X_test_mlp_proc = preprocessor_mlp_final.transform( X_test )

pred_test_mlp = modelo_mlp_final.predict( X_test_mlp_proc, verbose=0 ).ravel()

tiempo_inferencia_mlp = time.perf_counter() - inicio

mae_test_mlp = mean_absolute_error( y_test, pred_test_mlp )

RESULTADOS

resultados_test = pd.DataFrame({ "Modelo": [ "Baseline Media", "Baseline Lag 7", "XGBoost Optimizado", "MLP Optimizado" ], "MAE_Test": [ mae_test_media, mae_test_lag7, mae_test_xgb, mae_test_mlp ] }).sort_values( "MAE_Test" ).reset_index(drop=True)

print("TEST FINAL EVALUADO 📁") print(f"Observaciones: {len(y_test):,}") print( f"Periodo: {fechas_test.min().date()} " f"-> {fechas_test.max().date()}")

print("\nRESULTADOS:") display(resultados_test)

print("\nTIEMPO DE INFERENCIA") print( f"XGBoost: " f"{tiempo_inferencia_xgb:.6f} segundos" ) print( f"MLP: " f"{tiempo_inferencia_mlp:.6f} segundos" )

=====

BLOQUE DE CÓDIGO 64

```

```

=====

GUARDAR MODELOS FINALES

os.makedirs("../models", exist_ok=True)

XGBOOST

Guardamos pipeline completo:

preprocesamiento + modelo

ruta_xgb = "../models/xgboost_final.joblib"

joblib.dump( pipeline_xgb_final, ruta_xgb )

tamano_xgb_bytes = os.path.getsize(ruta_xgb)

MLP

Guardamos:

1. preprocesador

2. red neuronal

ruta_pre_mlp = "../models/preprocessor_mlp_final.joblib" ruta_mlp =
"../models/mlp_final.keras"

joblib.dump( preprocessor_mlp_final, ruta_pre_mlp )

modelo_mlp_final.save( ruta_mlp )

tamano_pre_mlp_bytes = os.path.getsize( ruta_pre_mlp )

tamano_mlp_bytes = os.path.getsize( ruta_mlp )

Tamaño total necesario para usar el MLP

tamano_mlp_total_bytes = ( tamano_pre_mlp_bytes + tamano_mlp_bytes )

CONVERSIÓN A KB

tamano_xgb_kb = tamano_xgb_bytes / 1024

tamano_pre_mlp_kb = tamano_pre_mlp_bytes / 1024 tamano_mlp_kb = tamano_mlp_bytes / 1024
tamano_mlp_total_kb = tamano_mlp_total_bytes / 1024

print("TAMAÑO EN DISCO") print("-" * 45)

print( f"XGBoost completo: " f"{tamano_xgb_kb:.2f} KB" )

print("\nMLP:") print( f" Preprocesador: " f"{tamano_pre_mlp_kb:.2f} KB" )

print( f" Red neuronal: " f"{tamano_mlp_kb:.2f} KB" )

print( f" TOTAL: " f"{tamano_mlp_total_kb:.2f} KB" )

=====

BLOQUE DE CÓDIGO 65

=====

```

GUARDADO DE LOS BASELINES

1. Baseline Lag 7

```
baseline_lag7 = { "nombre": "Baseline Lag 7", "variable": "lag_7", "regla": "prediccion =  
lag_7", "descripcion": ( "Utiliza como predicción el occupancy_rate observado " "del mismo  
barrio 7 días antes." ) }
```

```
ruta_lag7 = "../models/baseline_lag7.joblib"
```

```
joblib.dump( baseline_lag7, ruta_lag7 )
```

2. Baseline Media

```
baseline_media = { "nombre": "Baseline Media", "media_dev": float(media_dev), "regla":  
"prediccion = media_dev", "descripcion": ( "Utiliza como predicción constante la media de "  
"occupancy_rate calculada exclusivamente sobre Development." ) }
```

```
ruta_media = "../models/baseline_media.joblib"
```

```
joblib.dump( baseline_media, ruta_media )
```

Verificación

```
print("Archivos guardados correctamente:") print()
```

```
print( "Lag 7:", os.path.exists(ruta_lag7), "-", ruta_lag7 )
```

```
print( "Media:", os.path.exists(ruta_media), "-", ruta_media )
```

```
=====
```

BLOQUE DE CÓDIGO 66

```
=====
```

VERIFICACIÓN DE REPRODUCIBILIDAD DE LOS 4 ENFOQUES

1. XGBOOST

```
xgb_cargado = joblib.load( "../models/xgboost_final.joblib" )
```

```
pred_test_xgb_cargado = xgb_cargado.predict(X_test)
```

```
predicciones_iguales_xgb = np.allclose( pred_test_xgb, pred_test_xgb_cargado )
```

```
diferencia_max_xgb = np.max( np.abs(pred_test_xgb - pred_test_xgb_cargado) )
```

2. MLP

```
preprocessor_mlp_cargado = joblib.load( "../models/preprocessor_mlp_final.joblib" )
```

```
mlp_cargado = tf.keras.models.load_model( "../models/mlp_final.keras" )
```

```
X_test_mlp_cargado = preprocessor_mlp_cargado.transform(X_test)
```

```
pred_test_mlp_cargado = mlp_cargado.predict( X_test_mlp_cargado, verbose=0 ).ravel()
```

```
predicciones_iguales_mlp = np.allclose( pred_test_mlp, pred_test_mlp_cargado )
```

```
diferencia_max_mlp = np.max( np.abs(pred_test_mlp - pred_test_mlp_cargado) )
```

3. BASELINE LAG 7

```

pred_test_lag7_reproducido = X_test["lag_7"].to_numpy()

predicciones_iguales_lag7 = np.allclose( pred_test_lag7, pred_test_lag7_reproducido )

diferencia_max_lag7 = np.max( np.abs(pred_test_lag7 - pred_test_lag7_reproducido) )

4. BASELINE MEDIA

pred_test_media_reproducido = np.full( len(X_test), media_dev )

predicciones_iguales_media = np.allclose( pred_test_media, pred_test_media_reproducido )

diferencia_max_media = np.max( np.abs(pred_test_media - pred_test_media_reproducido) )

RESULTADOS

=====

print("VERIFICACIÓN DE REPRODUCIBILIDAD") print("=" * 40)

print("\n1. XGBoost") print(" Predicciones iguales:", predicciones_iguales_xgb) print("
Diferencia máxima:", diferencia_max_xgb)

print("\n2. MLP") print(" Predicciones iguales:", predicciones_iguales_mlp) print("
Diferencia máxima:", diferencia_max_mlp)

print("\n3. Baseline Lag 7") print(" Predicciones iguales:", predicciones_iguales_lag7)
print(" Diferencia máxima:", diferencia_max_lag7)

print("\n4. Baseline Media") print(" Valor de la media:", media_dev) print(" Predicciones
iguales:", predicciones_iguales_media) print(" Diferencia máxima:", diferencia_max_media)

=====

BLOQUE DE CÓDIGO 67

=====

Configuración ganadora de XGBoost

params_xgb_intervalo = { "n_estimators": 300, "max_depth": 3, "learning_rate": 0.05,
"subsample": 0.80 }

errores_validacion_xgb = []

for i, (fechas_train, fechas_val) in enumerate(folds_temporales, start=1):

# Máscaras temporales
mask_train_fold = fechas_dev.isin(fechas_train)
mask_val_fold = fechas_dev.isin(fechas_val)

X_train_fold = X_dev.loc[mask_train_fold]
y_train_fold = y_dev.loc[mask_train_fold]

X_val_fold = X_dev.loc[mask_val_fold]
y_val_fold = y_dev.loc[mask_val_fold]

# Copia del pipeline
modelo_fold = clone(pipeline_xgb)

# Configuración XGBoost seleccionada durante validación
modelo_fold.set_params(
    model__n_estimators=params_xgb_intervalo["n_estimators"],
    model__max_depth=params_xgb_intervalo["max_depth"],

```

```

        model__learning_rate=params_xgb_intervalo["learning_rate"],
        model__subsample=params_xgb_intervalo["subsample"]
    )

    # Entrenar solamente con fechas anteriores
    modelo_fold.fit(X_train_fold, y_train_fold)

    # Predecir período de validación
    pred_val = modelo_fold.predict(X_val_fold)

    # Error absoluto
    errores_fold = np.abs(
        y_val_fold.to_numpy() - pred_val
    )

    errores_validacion_xgb.extend(errores_fold)

errores_validacion_xgb = np.array(errores_validacion_xgb)

print("Observaciones de validación:", len(errores_validacion_xgb)) print(f"MAE:
{errores_validacion_xgb.mean():.6f}") print(f"Percentil 90:
{np.quantile(errores_validacion_xgb, 0.90):.6f}") print(f"Percentil 95:
{np.quantile(errores_validacion_xgb, 0.95):.6f}")

```